



Σavante

Lenguajes de programación

ÍNDICE

1. Introducción. Programa y algoritmos	5
2. Lenguajes de programación	6
2.1. Fases para realizar un programa	7
2.2. Tipos de lenguajes de programación	7
2.2.1. Lenguaje máquina	8
2.2.2. Lenguaje de bajo nivel (ensamblador)	8
2.2.3. Lenguajes de medio nivel	9
2.2.4. Lenguajes de alto nivel	10
2.3. Linker (enlazador)	11
2.4. Programas traductores	11
2.5. Abstracción en programación	13
2.6. Paradigmas de programación	15
3. Estructura de un programa	18
3.1. Instrucciones	19
3.1.1. Tipos de Instrucciones	19
3.2. Elementos básicos de un programa	21
3.3. Tipos de datos	22
3.3.1. Estructurados simples	23
3.3.1.1. Datos numéricos	23
3.3.1.2. Datos lógicos (booleanos)	23
3.3.1.3. Datos tipo carácter	23
3.3.2. Estructurados compuestos	24
3.4. Constantes y variables	26
3.5. Expresiones	28
3.5.1. Expresiones aritméticas	28
3.5.2. Expresiones lógicas (booleanas)	30
3.6. Escritura de algoritmos y programas	32
4. Control de flujo	33
4.1. Estructuras de control secuenciales	34

4.2. Estructuras de control selectivas	34
4.2.1. Estructuras selectivas simples (si-entonces/ if-then)	34
4.2.2. Estructuras selectivas dobles (si-entonces-sino/ if-then-else)	35
4.2.3. Estructuras de selección anidadas	36
4.2.4. Estructura selectiva múltiple (según-sea, caso-de/case)	36
4.3. Estructuras de control repetitivas o iterativas (bucles)	37
4.3.1. Estructura repetitiva mientras (WHILE)	38
4.3.2. Estructura repetitiva Repeat...Until	39
4.3.3. Estructura repetitiva para (For)	39
4.4. Estructuras de control de salto (jump)	40
5. Funciones y procedimientos	41
5.1. Declaración de funciones	43
5.2. Invocación de funciones	44
5.3. Parámetros formales y actuales	44
5.3.1. Parámetros. Paso por valor o por referencia	45
6. Recursividad	46
7. Principales lenguajes de programación	47
7.1. Lenguajes de programación más destacados	47
7.1.1. C	48
7.1.1.1. Características del lenguaje C	50
7.1.1.2. Entorno de C	51
7.1.1.3. Variables; declaración y asignación	52
7.1.1.3.1. Definir un array	55
7.1.1.4. Tokens de C	57
7.1.1.4.1. Palabras claves o reservadas (keywords)	57
7.1.1.4.2. Identificadores	57
7.1.1.4.3. Constantes	58
7.1.1.4.4. Operadores	58
7.1.1.4.5. Separadores	63
7.1.1.4.6. Comentarios	64

7.1.1.5. Agrupación de Tokens	65
7.1.1.6. Librerías	65
7.1.1.7. Estructura de un programa en C	67
7.1.1.8. Funciones de C	69
7.1.1.9. Memoria dinámica: fugas, dobles liberaciones y el infierno del HEAP	70
7.1.1.10. Interoperabilidad: cuando C se encuentra con Java/.NET (JNI/P/Invoke)	71
7.1.2. C++	72
7.1.2.1. Características del lenguaje C++	72
7.1.2.2. Sentencias en C++. Clasificación	73
7.1.2.3. Caso especial: la sentencia nula	78
7.1.2.4. Mecanismo de excepciones	79
7.1.2.5. Estructura de un programa en C++	80
7.1.3. C#	84
7.1.3.1. Estructura, Tipos y Operadores	85
7.1.3.2. Arrays, Colecciones y Métodos con Parámetros Opcionales	86
7.1.3.3. Clases, Modificadores y Patrón IDisposable	88
7.1.3.4. Excepciones y Logging Estructurado	90
7.1.3.5. Delegados, Lambdas y Expresiones LINQ Básicas	91
7.1.4. Java	93
7.1.5. Javascript	93
7.1.6. PHP	94
7.1.7. Python	94
7.1.7.1. Machine Learning Services para SQL Server con Python y R	100
7.1.7.2. ADOdb	101
7.1.8. Visual basic .Net (vb.Net)	102
7.1.9. Classic Visual Basic	103
7.1.10. SQL	103
7.1.11. PL/SQL	104
7.2. Otros lenguajes de programación	104
8. Generaciones de los lenguajes de programación	111
9. Bibliografía	112



1. Introducción. Programa y algoritmos



Imagen Programación (pixabay)

Antes de que empieces a aprender a programar, tienes que entender bien el concepto de Programa, Algoritmo y Estructuras de Datos.

Algoritmo

Un algoritmo es una sucesión de instrucciones ordenadas bien definidas (rutinas) que representa un modelo de solución a un determinado problema.

Unos datos de entrada seguirán esa sucesión de instrucciones para llegar una solución, obteniendo unos datos de salida.

Es una secuencia ordenada y precisa de instrucciones, pasos o procesos que permiten obtener unos resultados a partir de unos datos.

El algoritmo es independiente del lenguaje de programación y puede escribirse en lenguajes estándares como los diagramas de flujo y el pseudocódigo.

El algoritmo debe ser:

- **Muy claro.** Sin ambigüedad.
- **Finito.** Los recursos empleados deben ser finitos.
- **Independiente** del lenguaje que se emplee para implementar cada paso.



¿Qué es un Programa?

Un programa, es la traducción de un algoritmo a un lenguaje de programación, y puede estar formado por un conjunto de algoritmos, cada uno de los cuales lleve a cabo una tarea específica.

Es, por tanto, un conjunto ordenado de instrucciones, que controlan el ordenador para que realice una tarea específica.

Si un usuario tiene unas necesidades concretas, crearemos un programa (también se denomina aplicación). Que a través de unos procesos concretos podrá satisfacer esas necesidades del usuario. (Venta de productos, contabilidad...).

Para realizar esos procesos necesita utilizar datos que proporciona el usuario en el momento y/o que ya tiene almacenados. Estos datos están almacenados en la "Base de Datos".

Según Niklaus Wirth, un programa está formado por algoritmos y estructuras de datos.

Programa = Algoritmos + Estructuras de datos

Estructuras de datos

Las estructuras de datos están definidas por las relaciones entre los datos y la forma en que pueden agruparse.



Recuerda ver las clases emitidas en Temario Audiovisual

Las clases impartidas en directo y disponibles en Campus, en **Temario Audiovisual**, te ayudarán al entendimiento de la unidad, y además pueden tener información adicional.

[ACCEDE DIRECTAMENTE DESDE AQUÍ](#)

2. Lenguajes de programación

Son un medio de comunicación entre el usuario y el ordenador.

Un lenguaje de programación, es un conjunto de códigos (texto, símbolos...) que un ordenador interpretará (entenderá) y realizará las operaciones que dichos códigos le soliciten.



Cada lenguaje utilizará un determinado conjunto de símbolos (alfabeto), palabras (vocabulario) y reglas (semántica y sintaxis) para construir cada una de las órdenes o instrucciones que ejecutará el ordenador.

Un programa es un conjunto de órdenes o instrucciones, escritas en un lenguaje de programación, que resuelven un problema específico.

2.1. Fases para realizar un programa

El proceso de programación, está orientado a la resolución de un problema, lo dividimos en fases:

1. Definición y análisis del problema.
2. Diseño de algoritmos. Para ello usaremos diagramas de flujo o pseudocódigo.
3. Determinar el lenguaje de programación que utilizaremos basándonos los objetivos que debe cumplir.
4. Codificación (escritura) del programa en el lenguaje de programación elegido.
5. Depuración y verificación del programa. Prueba del programa con datos para comprobar que todo funcione correctamente, cumpla con sus objetivos, y no dé errores o se bloquee (se quede parado sin dar ningún mensaje ni continuar las secuencias de instrucciones).
6. Documentación (se irá realizando a lo largo de todos los pasos).
7. Mantenimiento. El usuario podrá querer modificar algunos procesos o tener nuevas necesidades. También habrá que realizar cambios necesarios si cambian leyes etc. (especialmente en programas de gestión: cambio del valor de IVA, IRPF...).

2.2. Tipos de lenguajes de programación

Existen varios tipos básicos de lenguajes de programación en función de la cercanía al lenguaje de la máquina o al del ser humano: su nivel de abstracción del procesador.

Esta abstracción se refiere al aislamiento de toda la información que no se necesita en un determinado nivel de conocimiento.

- Lenguaje máquina (lenguaje binario).
- Lenguaje de bajo nivel (ensamblador).
- Lenguajes de medio nivel.
- Lenguajes de alto nivel.



2.2.1. Lenguaje máquina

Fue el primer lenguaje de programación.

Las instrucciones se codifican como sucesiones de ceros y unos (bits, lenguaje binario), que es el único lenguaje que entiende el ordenador.

Cada procesador tiene un lenguaje máquina específico constituido por un repertorio reducido de operaciones. Por lo tanto, el lenguaje máquina depende directamente del *hardware*.

El lenguaje máquina es muy complicado, debemos escribir cadenas binarias, siendo muy fácil cometer errores al escribirlo y es poco eficiente a todos los niveles (escritura, interpretación, depuración y mantenimiento).

2.2.2. Lenguaje de bajo nivel (ensamblador)

Es un lenguaje orientado a máquina.

Consiste en asociar palabras clave a las operaciones del lenguaje máquina.

De esta forma, sustituimos los códigos binarios por códigos mnemotécnicos (más fáciles de identificar y recordar), normalmente en inglés.

Al sustituir operaciones del lenguaje máquina, estos lenguajes también dependen de cada procesador.

Normalmente se suele utilizar este tipo de lenguaje para programar los drivers (controladores) de los diferentes elementos de hardware del ordenador.



Ejemplo

- LOAD para cargar datos en una variable.
- ADD para sumar dos variables.
- STORE para guardar un valor en una variable.



Los programas no son directamente interpretados por el procesador, por lo que necesitaremos un programa traductor (ensamblador).

Los lenguajes de bajo nivel (y el lenguaje máquina que también podría considerarse de bajo nivel) tienen las siguientes ventajas y desventajas:

Ventajas

- Los programas ocupan poco espacio en memoria.
- Tiempo mínimo de ejecución.

Desventajas

- Dependen del procesador. Hay que conocer la arquitectura del hardware.
- Las operaciones son muy elementales, por lo que se requieren muchas instrucciones para realizar operaciones simples.
- El programador debe conocer un alto número de instrucciones.
- Es difícil de escribir, interpretar, depurar y mantener.

2.2.3. Lenguajes de medio nivel

Son lenguajes que, aunque ya se consideran de alto nivel, utilizan alguna metodología de los de bajo nivel.



Nota

El lenguaje C, siendo de alto nivel, utiliza propiedades de bajo nivel como acceder directamente a la memoria, por ello se considera también un lenguaje de medio nivel.

Son muy indicados para la creación de sistemas operativos, ya que no depende del hardware, pero sin embargo no pierden toda la eficiencia de los lenguajes de bajo nivel.



2.2.4. Lenguajes de alto nivel

Estos lenguajes no dependen del procesador ni de los componentes del ordenador, por lo que no necesitamos conocer el *hardware* de la máquina.

Las instrucciones (algoritmos) se expresan de forma más cercana a la capacidad cognitiva humana que al lenguaje de la máquina (bits).

Objetivos

- Buscar la mayor abstracción posible y facilitar la vida al programador, aumentando la productividad.
- Acercarse al lenguaje humano, facilitando así el entendimiento y trabajo del programador, y aumentando su productividad.

Ventajas

- Comprensibles. Están más próximos al lenguaje natural, instrucciones más sencillas y fáciles de comprender.
- Claros.
- Simples.
- Legibles.
- Eficientes.
- Sus instrucciones equivalen a múltiples instrucciones en lenguaje máquina.
- Son independientes del hardware, por lo que se pueden utilizar en diferentes ordenadores.

Desventajas

- Tienen que ser traducidos a lenguaje máquina para que el procesador los entienda (mediante un programa interprete o compilador).
- El nivel de detalle que un lenguaje de bajo nivel permite en la interacción con los componentes físicos, registros, memoria y operaciones suele ser mayor que el de los lenguajes de alto nivel. Sin embargo, los compiladores modernos pueden realizar optimizaciones que resultan ser más eficientes que el código escrito en ensamblador. Estas optimizaciones dependen en gran medida de la sofisticación del entorno de programación utilizado.



2.3. Linker (enlazador)

Pasa de un programa objeto a un programa ejecutable.

Es un programa que toma los objetos generados en los primeros pasos del proceso de compilación, la información de todos los recursos necesarios (biblioteca), quita aquellos recursos que no necesita, y enlaza el código objeto con su(s) biblioteca(s) con lo que finalmente produce un fichero ejecutable o una biblioteca. En el caso de los programas enlazados dinámicamente, el enlace entre el programa ejecutable y las bibliotecas se realiza en tiempo de carga o ejecución del programa.

2.4. Programas traductores

Los programas escritos en los distintos lenguajes deben ser traducidos a lenguaje máquina para que el ordenador pueda interpretarlas. Para ello existen diversos programas traductores que se pueden clasificar en:

- **Compiladores.**

El primer lenguaje traductor fue el "assembler" en español ensamblador, que traducía el lenguaje de programación Assembly (lenguaje de bajo nivel) en lenguaje máquina.

Por este motivo, la palabra "ensamblador" ha quedado en muchos casos como sinónima de "compilador". Pero no es lo mismo (un "ensamblador" solo compila lenguaje "Assembler").

Cada lenguaje de programación, tiene su propio lenguaje compilador.

Resumiendo, un "compilador" de XX Traduce las instrucciones de un "lenguaje XX" a instrucciones en lenguaje máquina.

Ejemplo: el lenguaje de compilación de C++ no será capaz de compilar un lenguaje escrito en JAVA.

La forma de trabajar de los distintos "compiladores" es diferente, y no todas tienen que pasar por todas las fases, pero podemos indicar como forma de trabajo más habitual la siguiente:

- Programa fuente: escrito en lenguaje de alto nivel.
- Programa objeto: es un archivo que ya está en lenguaje comprensible para la máquina.

Es la traducción realizada sobre el programa fuente (Código Fuente), pasándolo a un lenguaje máquina (propio del Sistema Binario).

Si no se requieren librerías, este programa objeto es ya el ejecutable.

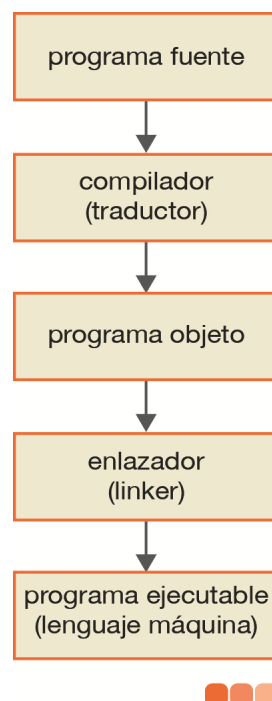


- Programa ejecutable: es el archivo que se ejecuta.

La diferencia principal entre el archivo objeto y el archivo ejecutable es que un archivo objeto es un archivo que se genera al compilar el código fuente, mientras que un archivo ejecutable es una mezcla del archivo objeto vinculándole un conjunto de librerías mediante un enlazador.

En el caso de no tener que enlazar librerías el objeto y el ejecutable serán el mismo.

Las fases de la compilación son las siguientes:



- **Intérpretes.**

Traducen de un lenguaje de alto nivel a lenguaje máquina, pero en lugar de generar un programa objeto, irán traduciendo y ejecutando instrucción a instrucción.

Este tipo de programa se usa principalmente para la depuración en la fase de programación, ya que es significativamente más lento que un programa compilado.

- **Preprocesadores.**

Incluimos este programa dentro del apartado de los traductores porque existe mucha literatura clásica de informática que así lo clasifica. Sin embargo, para ser exactos, no sería del todo correcto: en primer lugar, porque el resultado del archivo fuente tras su paso por él no está en lenguaje máquina, y en segundo lugar, porque tampoco realiza una traducción propiamente dicha, sino que se limita a añadir o suprimir determinadas partes del código.

Mantienen el lenguaje de alto nivel original, pero lo modifican textualmente para incorporar librerías, sustituir constantes, macros y aplicar condiciones. Completa el código antes del proceso de compilación.



El código siguiente formaría parte de un fichero fuente.

```
//fichero fuente

#include <stdio.h>
#define PI 3.14

int main() {
    printf("PI vale %.2f\n", PI);
}
```

A continuación, vemos el mismo código una vez ha pasado por el preprocesador.

```
int main() {
    printf("PI vale %.2f\n", 3.14);
}
```

Los preprocesadores en C eliminarían asimismo los comentarios.

2.5. Abstracción en programación

En informática, este concepto es muy usado como una manera de ocultar los detalles de implementación de un objeto, de forma que lo utilizamos, porque sabemos qué puede hacer, pero no necesitamos saber cómo lo hace. (*Convertimos un dato mediante la abstracción en un objeto*).



Ejemplo

En programación es muy común utilizar la función suma de dos números enteros. Sabemos qué obtendremos con la función suma, pero no necesitamos saber cómo el lenguaje de programación realiza la suma ni como la procesa el ordenador en lenguaje máquina.



En la abstracción realizamos dos procesos:

- Determinar los aspectos relevantes del problema en el nivel de abstracción que estamos estudiando.
- Ignorar los aspectos irrelevantes.

Muy importante, tener en cuenta que, un aspecto irrelevante en un determinado nivel de abstracción puede resultar relevante en otro nivel.

La abstracción permite al programador estudiar un problema complejo a través de un método jerárquico, se utiliza la abstracción para hacer los algoritmos más sencillos, a través de top-down.

En programación, un lenguaje de programación de alto nivel nos permite programar abstrayéndonos de los lenguajes de bajo nivel (como el ensamblador y lenguaje máquina) y de las particularidades del *hardware*.

En programación, la abstracción se enfoca de 2 formas:

- **Funcional.**

Es irrelevante cómo se realiza un proceso o acción y no importa su tiempo de ejecución.

Podemos indicar operaciones sin tener en cuenta el lenguaje de programación que utilizaremos posteriormente.

Indicamos, los datos de entrada, el proceso que se realiza, y el resultado que dará dicho proceso.

- **De datos.**

Se definen los tipos de datos, los valores que podrán tener, y las operaciones que podrán realizarse sobre ellos.

El tipo de dato es el primer nivel de abstracción, ya que no tenemos en cuenta cómo lo procesa la máquina. Gracias a esta abstracción, el sistema de tipos garantiza que cada dato solo pueda manipularse con las operaciones definidas para él, evitando que se produzcan incoherencias lógicas o errores graves durante la ejecución.

Con esto conseguimos que el programa no dependa de la máquina donde se ejecuta.

Podemos realizar diferentes niveles de abstracción al empezar a programar:

- **Abstracción Procedimental:** Se abstrae un conjunto preciso de operaciones como una operación simple.
- **Abstracción de Datos (TDA):** Tenemos un conjunto de datos y las operaciones que están vinculadas a los datos del tipo.
- **Abstracción de Iteración:** permite trabajar sobre colecciones de objetos sin tener que preocuparse por la forma concreta en que se organizan.



2.6. Paradigmas de programación

La evolución de los lenguajes de programación ha ido paralela a la idea de paradigma de programación.

Los paradigmas de programación, son estilos de desarrollos de programas, representan los diferentes enfoques para la construcción de un programa dependiendo del problema a resolver.

Los podemos clasificar según su manera de enfrentarse a los problemas y por tanto el tipo de soluciones que ofrecerá el programa creado.

Existen muchos tipos de paradigma, los más clásicos e importantes son:

- **Imperativo** (procedimental).

Representa el enfoque o método tradicional de programación.

Es un conjunto de instrucciones que se ejecutan una por una, de principio a fin, de modo secuencial, aunque este flujo puede ser modificado por instrucciones de salto o de control.

Este paradigma define el proceso de programación como el desarrollo de una secuencia de órdenes (comandos o instrucciones) que manipulan los datos para producir los resultados deseados.

Un lenguaje procedimental, se compone de un conjunto de sentencias que cambian su estado. Son secuencias de comandos (instrucciones) que ordenan acciones a la computadora, para que realice alguna tarea específica.

Cada instrucción es una orden u órdenes.

Ejemplos de lenguajes imperativos: COBOL, PASCAL, C, Ada y FORTRAN.

- **Declarativos.**

El paradigma declarativo, solicita al programador que describa el problema en lugar de encontrar una solución.

Un lenguaje declarativo utiliza el principio del razonamiento lógico para responder a las preguntas o cuestiones consultadas. Se basa en la lógica formal y en el cálculo de predicados de primer orden.

El razonamiento lógico se basa en la deducción.

Es opuesto al lenguaje Imperativo.

Ejemplos de lenguajes declarativos: Prolog, Haskell.



Dentro del paradigma de la programación declarativa podemos hacer una distinción de:

- Lógico:

Consiste en la aplicación de la lógica (basándose en los principios de demostración e inferencia válida).

La inferencia es el proceso por el cual se derivan conclusiones a partir de premisas (cada una de las proposiciones anteriores a la conclusión de argumento o razonamiento).

El problema se modela con enunciados de lógica de primer orden.

Lógica de primer orden, también llamada lógica predicativa, lógica de predicados o cálculo de predicados, es un sistema formal diseñado para estudiar la inferencia en los lenguajes de primer orden.

- Funcional:

Los programas se componen de funciones más matemáticas, es decir, implementaciones de comportamiento que reciben un conjunto de datos de entrada y devuelven un valor de salida.

- **Orientado a objetos.**

Es un enfoque totalmente distinto al imperativo, ya que el enfoque orientado a objetos guarda analogía con la vida real.

El desarrollo de *software* OO se basa en el diseño y construcción de objetos que se componen, a su vez, de datos y métodos (operaciones que manipulan esos datos).

El programador define en primer lugar los objetos del problema y, a continuación, los datos y operaciones que actuarán sobre esos datos.

Ejemplos de lenguajes orientados a objetos: Smalltalk, C++, Java, y VB.Net.

(Se basa principalmente en las técnicas: herencia, abstracción, polimorfismo y encapsulamiento, que estudiaras en su correspondiente unidad).

- **De aparición relativamente reciente.**

- Naturales:

Abreviado PLN, o NLP del idioma inglés Natural Language Processing.

Son los más actuales, se pretende acercar cada vez más las instrucciones del lenguaje de programación al lenguaje de las personas.

Se basa en la inteligencia artificial y lingüística que estudia las interacciones entre las computadoras y el lenguaje humano.



Algunos lenguajes y plataformas como **Wolfram Alpha** y **SQL** permiten consultas o interacciones que se acercan al lenguaje natural, pero todavía requieren una estructura específica que los distingue de un lenguaje completamente natural.

Estos lenguajes se considerarían 5GL.

- Orientado a datos, Lenguajes de desarrollo Rápido de Aplicaciones (RAD):

Estos paradigmas incluyen lenguajes de cuarta generación (4GL). Se caracterizan por su alto nivel de abstracción, facilidad de uso y productividad. El objetivo de estos lenguajes es reducir la cantidad de código que debe desarrollarse, maximizando la eficacia en la creación de aplicaciones complejas, como bases de datos y generación de informes.

Algunos ejemplos son SQL, MATLAB, ABAP, Informix-4GL.

- Dirigido por eventos:

El flujo del programa está determinado por sucesos externos (por ejemplo, una acción del usuario).

Es un paradigma de programación en el que tanto la estructura como la ejecución de los programas van determinados por los sucesos que ocurran en el sistema, definidos por el usuario o que ellos mismos provoquen.

Mientras en la programación secuencial (o estructurada) es el programador, el que define cuál va a ser el flujo del programa, en la programación dirigida por eventos, será el propio usuario (o lo que esté accionando el programa), el que dirija el flujo del programa.

Puede haber intervención de un agente externo al programa, en cualquier momento.

Si bien el paradigma de la programación dirigida por eventos puede considerarse antiguo y está vinculada a lenguajes de 3GL es un tipo de enfoque que sigue siendo usado cotidianamente en distintos frameworks JavaScript como React, Angular o Vue.js.

- Orientado a aspectos:

La Programación Orientada a Aspectos o POA (en inglés AOP: Aspect-Oriented Programming) es un permite una adecuada modularización de las aplicaciones y posibilita una mejor separación de responsabilidades (Obligación o correspondencia de hacer algo).

Apunta a dividir el programa en módulos independientes, cada uno con un comportamiento bien definido.

Gracias a la POA se pueden encapsular los diferentes conceptos que componen una aplicación en entidades bien definidas, eliminando las dependencias entre cada uno de los módulos. De esta forma se consigue razonar mejor sobre los conceptos, se elimina la dispersión del código y las implementaciones resultan más comprensibles, adaptables y reusables.

Son lenguajes que aparecen en la 3GL, pero que siguen de actualidad hoy día en lenguajes como C#, Java, Javascript.



Atención

Lenguaje multiparadigma:

Es aquel que utiliza más de un paradigma de programación.

Por ejemplo, Python es un lenguaje orientado a objetos, reflexivo, imperativo y funcional.

3. Estructura de un programa

Vamos a estudiar el concepto de programa como un conjunto de instrucciones y los tipos de instrucciones.

El programador debe especificar tres bloques:

- **Entradas.**

Es el proceso de introducir la información de entrada (datos) en la memoria del ordenador, operación de lectura o acción de leer.

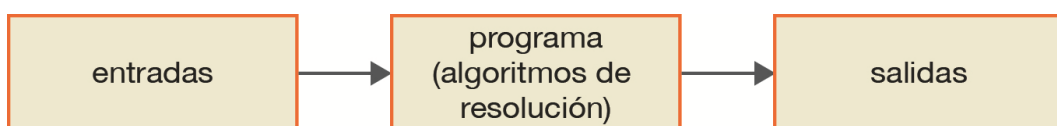
- **Algoritmos de resolución.**

Son el código que transforma las entradas (datos) en las salidas (resultados).

- **Salidas.**

Se deben presentar en un dispositivo de salida, como una pantalla, impresora o dispositivo de almacenamiento de información.

La operación de salida de datos se conoce también como escritura o acción de escribir.



Bloques de un programa



3.1. Instrucciones

El proceso de diseño del algoritmo o posteriormente de codificación del programa consiste en definir las acciones o instrucciones que resolverán el problema.

Un programa puede ser:

- **Lineal.**

Las instrucciones se ejecutan secuencialmente (sin bifurcaciones, decisiones ni comparaciones).

- **No lineal.**

Cuando se interrumpe la secuencia mediante instrucciones de bifurcación.

3.1.1. Tipos de Instrucciones

Las instrucciones disponibles en un lenguaje de programación dependen del tipo de lenguaje.

Vamos a indicar las instrucciones (acciones) básicas que normalmente están soportadas por todos los lenguajes, que son:

1. Instrucciones de inicio/fin.
2. Instrucciones de asignación.
3. Instrucciones de lectura.
4. Instrucciones de escritura.
5. Instrucciones de bifurcación.

1). Instrucciones de inicio/fin

Se utilizan para indicar el comienzo y final del algoritmo.

Tipo de instrucción	Pseudocódigo inglés	Pseudocódigo español
comienzo de proceso	begin	inicio
fin de proceso	end	fin



2). Instrucciones de asignación

Tipo de instrucción	Pseudocódigo
asignación	$A \leftarrow B$

La asignación consiste en asignar un valor a una variable.

Por ejemplo:

$A \leftarrow 30$ asigna el valor 30 a la variable A.



Reto

Vamos a proponerte un reto muy sencillo. Debes averiguar el valor que tendrá la variable C al final.

- $A \leftarrow 15.$
- $B \leftarrow 18.$
- $C \leftarrow A.$
- $B \leftarrow A+2.$
- $C \leftarrow B.$

Solución:

17.

3). Instrucciones de lectura de datos (entrada)

Esta instrucción lee datos de un dispositivo de entrada (teclado, ratón, disco, etcétera).

Por ejemplo, si escribimos la instrucción: **leer (A, B, C).**

Y a continuación introducimos por teclado los números 1, 2 y 3, se asignarían los siguientes valores: **A =1, B=2, C=3.**



Sería equivalente a escribir:

- $A \leftarrow 1.$
- $B \leftarrow 2.$
- $C \leftarrow 3.$

4). Instrucciones de escritura

Estas instrucciones muestran los valores en un dispositivo de salida. Si escribimos las siguientes instrucciones:

- $A \leftarrow 1.$
- $B \leftarrow 2.$
- $C \leftarrow 3.$
- escribir (A, B, C).

Nos mostraría por el dispositivo de salida (pantalla, impresora, etcétera) los valores 1, 2 y 3.

5). Instrucciones de bifurcación

Una bifurcación interrumpe la secuencia lineal de un programa.

Hay dos tipos:

- **Incondicional:** la bifurcación se realiza siempre que el flujo del programa pase por la instrucción sin necesidad del cumplimiento de ninguna condición.
- **Condicional:** la bifurcación se realiza de modo condicional en función del resultado de la evaluación de una condición.

3.2. Elementos básicos de un programa

Los lenguajes de programación tienen elementos básicos que se utilizan como bloques constructivos, así como reglas para que esos elementos se combinen.

Estas reglas se denominan sintaxis del lenguaje. Solamente las instrucciones sintácticamente correctas pueden ser interpretadas por la computadora; los programas que contengan errores de sintaxis son rechazados por la máquina.



Los elementos básicos constitutivos de un programa o algoritmo son:

- Palabras reservadas (inicio, fin, si-entonces, etcétera).
- Identificadores (nombres de variables, procedimientos, funciones, programas, etcétera).
- Caracteres especiales (coma, apóstrofo, etcétera).
- Constantes.
- Variables.
- Expresiones.
- Instrucciones.

Otros elementos importantes son:

- Bucles.
- Contadores.
- Acumuladores.
- Interruptores.
- Estructuras:
 - Secuenciales.
 - Selectivas.
 - Repetitivas.

3.3. Tipos de datos

Un programa de computadora opera sobre datos. En los lenguajes de programación los datos deben ser de un tipo de dato específico.

El tipo de datos indica el conjunto específico de valores que pueden tomar los datos y las operaciones que pueden actuar sobre estos datos.

Existen dos tipos de datos:

- Estructurados Simples.
- Estructurados Compuestos.



3.3.1. Estructurados simples

Los tipos de datos simples o primitivos son aquellos que no están compuestos por otros tipos de datos.

Los tipos de datos simples básicos son los siguientes:

- Numéricos (entero, real).
- Lógicos (booleanos).
- Carácter.

Sin embargo, los lenguajes de programación pueden admitir otros tipos de datos e incluso permitir al programador definir los suyos propios.

3.3.1.1. Datos numéricos

Hay dos tipos de datos numéricos básicos:

- **Entero (*Integer*):** aquellos que no tienen parte fraccionario ni decimales. Pueden ser negativos o positivos.
- **Real:** los números reales tienen una parte decimal y pueden ser positivos o negativos. Un número real consta de un entero y una parte decimal.

3.3.1.2. Datos lógicos (booleanos)

Es aquel que puede tomar uno de dos valores (verdadero o falso, 0 o 1, sí o no, etcétera). Se utilizan en estructuras condicionales.

3.3.1.3. Datos tipo carácter

Un dato tipo carácter contiene un solo carácter. Los caracteres que reconocen las diferentes computadoras no son estándar.

Sin embargo, la mayoría reconoce los siguientes caracteres alfabéticos y numéricos:

- Caracteres alfabéticos (A, B, C, ..., Z), (a, b, c, ..., z).
- Caracteres numéricos (1, 2, ..., 9, 0).
- Caracteres especiales (+, -, *, /, ^, etcétera).



El experto opina

Algunos autores definen la cadena de caracteres como un tipo de dato simple.

Es cierto que la mayoría de los lenguajes lo tienen como variable básica, pero en realidad es una estructura formada por varios elementos de tipo carácter.

3.3.2. Estructurados compuestos

Una estructura de datos es una colección de datos que pueden ser caracterizados por su organización y las operaciones que se definen en ella.

Los tipos de datos compuestos se construyen con tipos de datos primitivos. Los tipos de datos compuestos se pueden organizar en diferentes estructuras de datos.

Los tipos de datos más frecuentes utilizados en los diferentes lenguajes son:

Tipo de Dato	Java	C++	PHP	Python	Javascript
Arrays (vectores/matrices)	Estático, tamaño fijo	Estático (tamaño fijo) / Dinámico (con std:vector)	Dinámico	Dinámico	Dinámico
Cadenas	Clase String Inmutable	Clase std::string Mutable	Tipo string mutable	Tipo str mutable	Tipo primitivo string Mutable
Registros	Clases o struct	struct	arrays asociativos	clases / namedtuple de collection	Pueden usarse objetos
Conjuntos	Hashset	Clase std::set	Array asociativo	set	Set (ECMAScript 2015)
Ficheros	File	Soporte con bibliotecas de entrada/salida	Funciones, p.e fopen/ fwrite	Funciones open, read, write...	File API

Esta imagen es un ejemplo de varios lenguajes de programación, la configuración de los tipos de datos no siempre es la misma.



Estáticas

El tamaño ocupado en memoria se define antes de que el programa se ejecute y no puede modificarse dicho tamaño durante la ejecución del programa.



Aviso

Ya has estudiado la mayoría de estas estructuras en temas anteriores. Por eso, vamos a estudiar sólo:

Tipos de Datos Estructurados, Compuestos, Estáticos: Arrays y Registros.

- **Arrays.**

Como vimos en la tabla, en algunos lenguajes un array puede ser una estructura de datos homogénea (todos sus datos del mismo tipo).

Un *array* en Java o C es una estructura de datos que almacena un conjunto de valores, todos del mismo tipo de datos, cuyo acceso se realiza por índices. El índice de un elemento (1, 2, ..., i, ..., n) designa su posición en la ordenación del vector.

Algunos lenguajes empiezan por cero en lugar de por uno. De esta forma, las posiciones serían: (0, 1, 2, ..., i, ..., n-1).

Hay dos tipos de *arrays*: Vectores y matrices:

- **Vector.**

Es un *array* unidimensional. **Los vectores** se almacenan en la memoria central de la computadora en orden adyacente. Las operaciones que se pueden realizar con vectores son:

- » Asignación.
- » Lectura/escritura.
- » Recorrido (acceso secuencial).
- » Actualizar (añadir, borrar, insertar).
- » Ordenación.
- » Búsqueda.

- **Matriz.**

Es un *array* multidimensional. Una matriz de dos dimensiones es un vector de vectores.



- **Registro.**

Un registro es una estructura de datos heterogénea (cada componente puede ser de un tipo de datos distinto).

Registro es un tipo de dato estructurado que almacena varios tipos de elementos bajo una misma variable.



Recuerda

La diferencia entre registro y array es la clase de elementos que contienen.

- Un *array* es una estructura de datos homogénea (todos sus datos son del mismo tipo).
- Un registro es una estructura de datos heterogénea (cada componente puede ser de un tipo de datos distinto).

Dinámicas

No tienen restricciones de tamaño o memoria ocupada.

3.4. Constantes y variables

Veamos la diferencia:

- **Constante:**

Es un dato que permanece sin cambios durante todo el desarrollo del algoritmo o durante la ejecución del programa.

- **Variable:**

Es un objeto o tipo de datos cuyo valor puede cambiar durante el desarrollo del algoritmo o ejecución del programa.

Dependiendo del lenguaje, hay diferentes tipos de variables, tales como enteras, reales, carácter, lógicas y de cadena.



Una variable que es de un cierto tipo puede tomar únicamente valores de ese tipo. Si se intenta asignar un valor de un tipo a una variable de otro tipo, se producirá un error de tipo.

Una variable se identifica por los siguientes atributos:

- **Nombre o identificador.**

Lo identifica:

- Es un conjunto de caracteres alfanuméricos que suelen empezar por una letra.
- Se debe evitar utilizar palabras reservadas del lenguaje de programación.
- Los nombres de las variables deben ser significativos y tener relación con el objeto al que representan.

- **Tipo.**

Describe el uso de la variable.

Declaración de constantes y variables

Por norma general, las variables y constantes deben ser declaradas antes de ser utilizadas.

- **Constantes.**

Sintaxis de la declaración:

```
const <tipo_de_dato> <nombre_constante> = <expresión>
```

Ejemplo: const doble PI = 3.141592.

- **Variables.**

Sintaxis de la declaración:

```
<tipo_de_dato> <nombre_variable> [= <expresión>]
```

Ejemplo: entero edad = 23.



3.5. Expresiones

Hay 2 tipos de expresiones:

- **Aritméticas.**
 - Operando.
 - Operador.
- **Lógicas o booleanas.**
 - De relación.
 - Operadores lógicos.

Además de conocerlas, es muy importante conocer las reglas de prioridad en que se ejecutan.

3.5.1. Expresiones aritméticas



Fuente: Pixabay

Las expresiones aritméticas son análogas a las fórmulas matemáticas. En una expresión aritmética hay dos tipos de objetos:

- **Operando.**
Los números que participan en la operación.
- **Operador.**
Indica la operación que se realizará sobre los operandos.



Ejemplo:

- $A + 3$.
- A y 3 son números (A será una variable numérica).
- $+$ es el operador y nos indica que va a sumar los dos números.

No todos los operadores existen en todos los lenguajes. A continuación, mostramos una tabla con los más importantes:

Operador	Significado
$+$	Suma
$-$	Resta
$*$	Multiplicación
$/$	División
div	División entera
mod	Módulo
$++$	Incremento
$--$	Decremento
$^$	Exponenciación

Reglas de prioridad

Son reglas matemáticas que permitan determinar el orden de las operaciones.

Las reglas de prioridad o precedencia determinan el orden de las operaciones cuando una expresión tiene dos o más operandos. Estas son:

- Las operaciones que están encerradas entre paréntesis se evalúan primero.

Si existen diferentes paréntesis anidados (interiores unos a otros), las expresiones más internas se evalúan primero.

- Las operaciones aritméticas dentro de una expresión siguen el siguiente orden de prioridad:

1. Paréntesis ():

Permiten agrupar expresiones y forzar su evaluación antes que el resto de operadores.



2. Operadores unarios

- » $+$ (operador unario positivo), no altera el valor del operando.
- (operador unario negativo), cambia el signo del operando
- » $++$ y $---$ (operadores de incremento y decremento, aumentan o disminuyen el valor del operando en una unidad; pueden utilizarse en forma prefija o postfija, siendo la postfija de mayor prioridad que la prefija).

3. Exponenciación ($**$)

Operador que eleva un valor a la potencia indicada por otro.

En JavaScript y PHP, el operador de exponenciación $**$ tiene mayor prioridad que los operadores unarios $+$ y $-$, por lo que expresiones como $-2 ** 2$ se interpretan como $-(2 ** 2)$.

4. Multiplicación, división y resto: $*$, $/$, $\%$

Operadores que realizan el producto, la división y el cálculo del resto de una división entera. Todos ellos tienen el mismo nivel de prioridad y se evalúan de izquierda a derecha.

5. Adición y sustracción: $+$, $-$

Operadores que realizan la suma y la resta. En JavaScript, el operador $+$ también puede actuar como concatenación de cadenas cuando alguno de los operandos es de tipo texto.

Este orden de prioridad es una guía didáctica general. La precedencia y asociatividad de los operadores dependen del lenguaje y de su especificación; cuando varios operadores tienen la misma prioridad, el orden de evaluación lo marca su asociatividad, normalmente de izquierda a derecha. Ante expresiones complejas, deben consultarse las reglas del lenguaje utilizado.

3.5.2. Expresiones lógicas (booleanas)



Fuente: Pixabay

El resultado de una expresión lógica es siempre verdadero o falso. Las expresiones lógicas se forman combinando constantes lógicas y variables lógicas con operadores lógicos y/o relacionales.



Operadores de relación:

Los operadores relacionales o de relación permiten realizar comparaciones de valores de tipo numérico o carácter. Los operadores de relación sirven para expresar las condiciones en los algoritmos.

El formato general para las comparaciones es:

- Expresión1 operadorDeRelación expresión2.

Los principales operadores de relación son:

Operador	Significado
<	Menor que
>	Mayor que
=, ==	Igual que
<=	Menor o igual que
>=	Mayor o igual que
< >, !=	Distinto de

Operadores lógicos:

Los operadores lógicos o booleanos básicos son NOT (no), AND (y) y OR (o). A continuación, te mostramos una tabla con su funcionamiento:

A	B	NOT A	A AND B	A OR B
Verdadero	Verdadero	Falso	Verdadero	Verdadero
Verdadero	Falso	Falso	Falso	Verdadero
Falso	Verdadero	Verdadero	Falso	Verdadero
Falso	Falso	Verdadero	Falso	Falso

Por lo tanto:

- NOT A. Es verdadero cuando A es falsa y viceversa.
- A AND B. Es verdadero cuando A y B son verdaderas.
- A OR B. Es verdadero cuando alguna de las dos es verdadera (también si lo son las dos).



Reglas de prioridad

Se ejecutan en el siguiente orden:

- NOT.
- AND.
- OR.
- Operadores de relación.

Si se utilizan paréntesis, estos siempre tendrán prioridad.

3.6. Escritura de algoritmos y programas

La escritura de un algoritmo mediante una herramienta de programación debe ser lo más clara posible y estructurada, de modo que su lectura facilite considerablemente el entendimiento del algoritmo y su posterior codificación en un lenguaje de programación.

Un algoritmo consta de dos componentes:

- **Una cabecera de programa.**

La cabecera del programa es la palabra "algoritmo" seguida del nombre del algoritmo.

- **Un bloque de algoritmo.**

Es el resto del programa y consta de dos componentes o secciones:

- **Las acciones de declaración.**

Las declaraciones definen o declaran las variables y constantes que tengan nombres.

- **Las acciones ejecutables.**

Son las acciones o instrucciones que deberá realizar el ordenador cuando se ejecute un programa.

Ejemplo de estructura de Algoritmo

En este ejemplo usaremos "//" para indicar que es un comentario. Existen diversas formas de indicar comentarios, las cuales dependen del lenguaje de programación utilizado.



Algoritmo ejemplo:

```
// Cabecera del programa. Aquí se podrían definir lo que hace el algoritmo, las
// entradas y las salidas.

//Sección de declaración
var
entera : N
real : A, B
const
pi = 3.1416
//Sección de acciones
Inicio
Leer(N)
Leer(A,B)
Escribir (N*2)
Escribir (A+B)
fin
```

4. Control de flujo

El término **control de flujo** se refiere al orden en que se ejecutan las sentencias en un programa.

A menos que se especifique expresamente, el control de flujo normal de un programa es secuencial, es decir, que las sentencias se ejecutan una detrás de otra en el orden en el que están escritas en el programa.

Las estructuras de control de flujo permiten que el flujo secuencial del programa sea modificado de un modo preciso y definido con anterioridad.

Cualquier algoritmo, no importa su complejidad, puede ser construido utilizando combinaciones de las estructuras de control de flujo estandarizadas.

Las estructuras de control son:

- Secuenciales.
- Selectivas.



- Repetitivas o iterativas.
- De salto (*jump*).

4.1. Estructuras de control secuenciales

Una estructura secuencial es aquella en la que una acción (instrucción) sigue a otra en secuencia.

Las tareas se suceden de tal manera que la salida de una es la entrada de la siguiente y así sucesivamente hasta el final del proceso. La estructura secuencial tiene una entrada y una salida.

4.2. Estructuras de control selectivas

Las estructuras selectivas se utilizan para tomar decisiones lógicas cuando existen un número de posibles alternativas resultantes de la evaluación de determinada condición.

La representación de una estructura selectiva se hace con instrucciones de pseudocódigo (*if, then, else*; en español, si, entonces, sino), con una figura geométrica de rombo, o bien con un triángulo en el interior de una caja rectangular.

Las estructuras selectivas pueden ser:

- Simples (si-entonces/ *if-then*).
- Dobles (si-entonces-sino/ *if-then-else*).
- Anidadas (un *if* dentro de otro).
- Múltiples (según-sea, caso-de/ *case*).

Vamos a estudiar cada una de las cuatro, con más detenimiento.

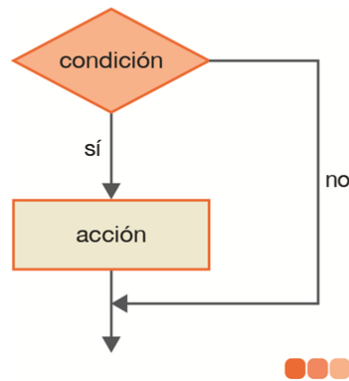
4.2.1. Estructuras selectivas simples (si-entonces/ *if-then*)

La selección si-entonces evalúa la condición y:

- Si la condición es verdadera, entonces ejecuta la acción.
- Si la condición es falsa, entonces no hace nada.



Diagrama de flujo:



Pseudocódigo:

si <condición> **entonces**

<acción 1>

fin_si

4.2.2. Estructuras selectivas dobles (si-entonces-sino/ if-then-else)

La estructura selectiva doble permite elegir entre dos alternativas posibles.

Se evalúa la condición y:

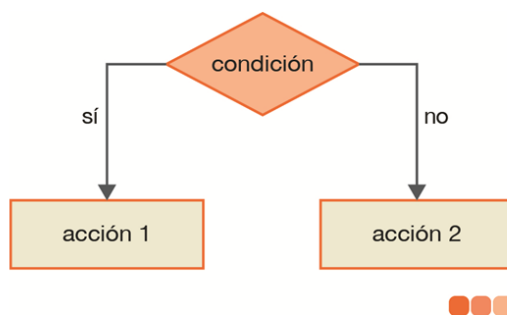
- Si la condición es verdadera (*if/si*).

Entonces se ejecuta la acción 1.

- Si la condición es falsa (*else/si-no*).

Entonces se ejecuta la acción 2.

Diagrama de flujo:



Pseudocódigo:

si <condición> **entonces**

<acción 1>

si_no

<acción 2>

fin_si

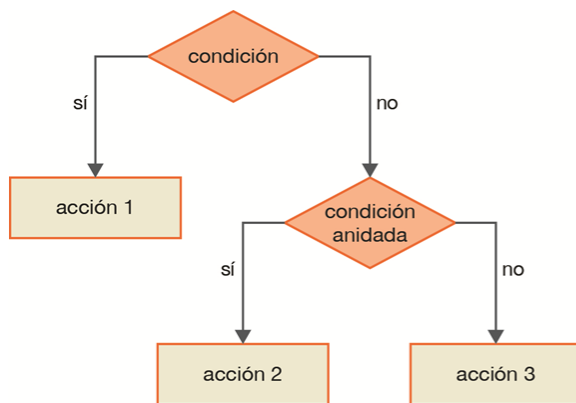


4.2.3. Estructuras de selección anidadas

Cuando hay más de dos opciones que escoger, se pueden anidar estructuras simples o dobles, es decir, una estructura simple o doble puede contener otra estructura simple o doble y así hasta agotar las posibles opciones.

Las estructuras *si/if* interiores a otras estructuras *si/if* se denominan anidadas o encajadas. Es recomendable utilizar indentación (sangría o sangrado) al escribir el código.

Diagrama de flujo:



Pseudocódigo:

```

si <condición> entonces
  <acción 1>
si_no
  si <condición anidada> entonces
    <acción 2>
  si_no
    <acción 3>
fin_si
  
```

4.2.4. Estructura selectiva múltiple (según-sea, caso-de/case)

En el caso de que haya más de dos opciones posibles, se podría resolver el problema anidando estructuras simples o dobles.

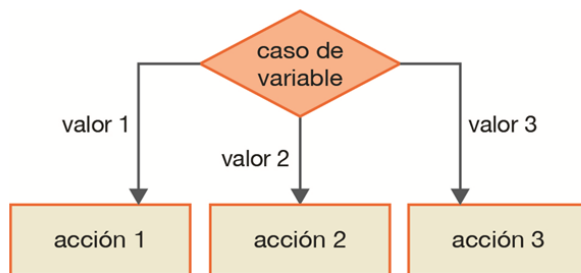
Sin embargo, si las alternativas son muchas, esto puede provocar problemas de legibilidad.

La estructura selectiva múltiple evaluará una expresión que podrá tomar n valores distintos 1, 2, 3, ..., n .

Según se elija uno de estos n valores en la condición se realizará una de las n acciones, o lo que es igual, el flujo del algoritmo seguirá un determinado camino entre los n posibles.



Diagrama de flujo:



Pseudocódigo:

Caso de (Expresión)

Caso Valor1

Acción1

Case Valor2

Acción2

Case Valor3

Acción3

Case Else

// (Bloque de sentencias a ejecutar
si no hay un *case* para el valor)

End Select

4.3. Estructuras de control repetitivas o iterativas (bucles)

Son estructuras de control que permiten repetir una secuencia de instrucciones mientras se cumpla una determinada condición. El número de repeticiones no suele estar fijado previamente, sino que depende de la evaluación de dicha condición durante la ejecución del programa.

Estas estructuras, denominadas bucles, forman parte de los elementos fundamentales de control del flujo de un programa, junto con las estructuras secuenciales y selectivas. La condición que controla la repetición puede evaluarse al inicio o al final del bucle.

Cada ejecución completa del conjunto de instrucciones que se repiten recibe el nombre de iteración.

Los tres bucles (estructura repetitiva o iterativa) más utilizados son:

bucle **While**, bucle **Repeat... Until** y el bucle **For**



4.3.1. Estructura repetitiva mientras (WHILE)

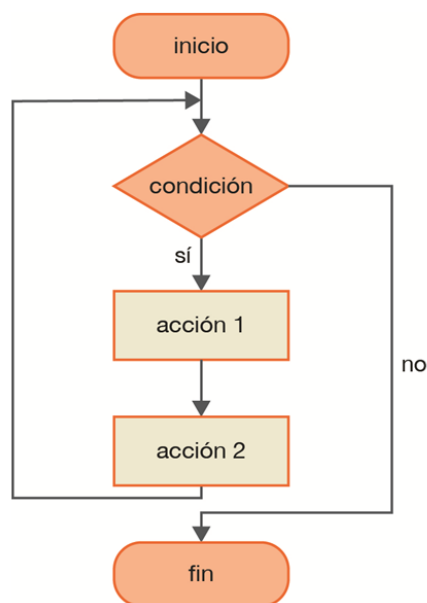
Hay 2 tipos, dependiendo de si la condición de parada se realiza al principio del bucle o se origina al final del bucle.

- **La condición de parada se realiza al principio del bucle.**

Estructura WHILE (repetir-mientras)

Se ejecuta el bucle mientras se verifique la condición inicial. Puede darse el caso de que el bucle no se ejecute ninguna vez.

Diagrama de flujo:



Pseudocódigo:

```

mientras (condición) {
    ...
    Instrucciones
    ...
}
  
```

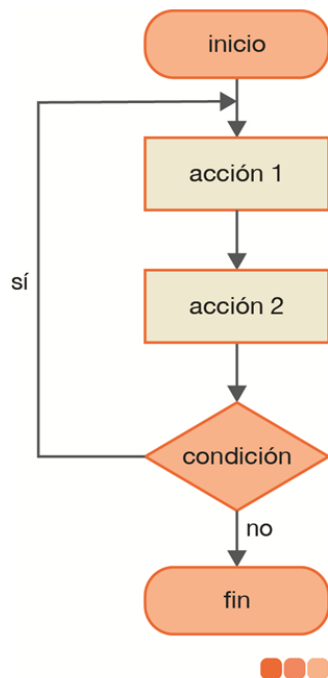
- **La condición de parada se origina al final del bucle:**

Estructura DO-WHILE (repetir-mientras)

El bucle se ejecuta hasta que se verifique la condición. Al estar la condición de parada al final del bucle, nos aseguramos de que el bucle se ejecute al menos una vez.



Diagrama de flujo:



Pseudocódigo:

```

hacer {
    ...
    Instrucciones
    ...
} mientras (condición);
  
```

4.3.2. Estructura repetitiva Repeat...Until

Similar al bucle While.. en su segundo tipo (DO-WHILE), la condición de parada se ejecuta al final del bloque.

La expresión booleana va después de la ejecución del bucle, siempre se ejecuta la instrucción como mínimo una vez, y se repite hasta que el resultado de la condición sea "verdadero".

Diferencias con el bucle While:

El bucle Repeat...Until siempre se ejecuta por lo menos una vez, mientras que, While puede no ejecutarse ninguna vez si la expresión booleana es inicialmente falsa.

4.3.3. Estructura repetitiva para (For)

La condición de salida del bucle se realiza con un contador que indica el número de iteraciones, la instrucción se repite el número de veces determinado que indicamos.

Hay que indicar un valor de entrada, una condición y un incremento o decremento, en el siguiente orden:

- Para (inicialización; condición; incremento o decremento).



Ejemplo:

```
Para (i=1;i<10;i=i+1){  
    instrucciones  
}
```

Este bucle se repetirá 9 veces: para i igual a uno, mientras i sea menor de 10, y en cada pasada se suma 1 a i.

4.4. Estructuras de control de salto (jump)

Hay instrucciones que "causan un salto", alteran el flujo del programa.

Hay principalmente dos formas de instrucción de salto:

- **El salto incondicional.**

Siempre se realiza.

- **El salto condicional.**

Puede ser efectivo o no según una determinada condición, como por ejemplo el contenido de algún registro de la CPU.

Comportamiento del programa según se produzca o no el salto:

- **El salto resulta efectivo.**

La siguiente instrucción ejecutada será aquella marcada como destino del salto.

- **El salto no es efectivo.**

El flujo de programa no cambia, por tanto, se ejecuta inmediatamente la siguiente instrucción en el código.



5. Funciones y procedimientos

Divide y Vencerás



Fuente: Pexels

Un método muy recomendable para solucionar un problema complejo es dividirlo en problemas más sencillos, y seguir descomponiendo hasta que los problemas sean fáciles de resolver, se le denomina divide y vencerás.

El problema principal, se soluciona por el correspondiente programa o algoritmo principal, y la solución de los subproblemas, mediante subprogramas, conocidos como procedimientos (subrutinas) o funciones.

Los subprogramas, cuando se tratan en lenguaje algorítmico, se denominan también subalgoritmos.

Un subprograma puede realizar las mismas acciones que un programa:

- Aceptar datos.
- Realizar cálculos.
- Devolver resultados.

Un subprograma es utilizado por el programa para un propósito específico. El subprograma recibe datos desde el programa y le devuelve resultados. Se dice que el programa principal llama o invoca al subprograma.

El subprograma ejecuta una tarea y, a continuación, devuelve el control al programa al punto desde donde se realizó la llamada. Un subprograma puede llamar a su vez a sus propios subprogramas.



Funciones y procedimientos

Matemáticamente, una función es una operación que toma uno o más valores (llamados argumentos) y produce un valor denominado resultado.

En programación:

- **función** es un bloque de código identificado por un nombre que agrupa instrucciones para realizar una tarea específica, devolviendo un valor al final.

```
//ejemplo de función en PHP
function suma($num, $num2) {
    $resultado = $num + $num2;
    return $resultado;
}
echo suma(5,3);
```

- **procedimiento** puede considerarse como una función que no devuelve valor (void en C, None en Python).

```
//ejemplo de procedimiento en PHP
function saludo ($nombre) {
    echo "Hola $nombre";
}
saludo("Pedro");
```

Sin embargo, en lenguajes clásicos como Pascal o PL/SQL se hace una distinción formal: las funciones devuelven valor y los procedimientos no, aunque ambos sean bloques de código invocables. Así pues según la definición clásica el procedimiento: no devuelve un valor como tal, pero sí puede producir efectos en las variables del programa llamador mediante parámetros por referencia, o realizando acciones como imprimir, escribir en un archivo, etc.

Todos los lenguajes de programación tienen funciones incorporadas y funciones definidas por el usuario.

Cada lenguaje de programación tiene sus propias funciones incorporadas. Para utilizarlas, se escribe el nombre de la función, con, eventualmente los argumentos adecuados.



Por ejemplo: **cos**(x) calcularía el coseno del valor que tenga x.

A una función no se la llama explícitamente, sino que se invoca o referencia mediante un nombre (de dicha función) y, si así se requiere, una lista de parámetros actuales.

5.1. Declaración de funciones

La declaración de una función requiere una serie de pasos que la definen.

Una función tiene una constitución similar a los algoritmos.

Por consiguiente, constará de:

- **Cabecera**, dependiendo de los lenguajes podemos encontrarnos distintos tipos de cabeceras:
 - nombre + paréntesis **dado()**
 - tipo de retorno + nombre + paréntesis, **int suma(a, b)**
 - palabra clave (function o equivalente) + nombre + paréntesis, **function(num)**

Entre los paréntesis que siguen al nombre podemos encontrar los llamados parámetros formales o argumentos, que veremos a continuación.

- **Cuerpo**: constará de una serie de acciones o instrucciones cuya ejecución determinará el valor de salida, si lo tiene, de la función. En algunos lenguajes este valor se asigna al nombre de la función (ej. Pascal), mientras que en la mayoría se especifica mediante la sentencia return, que devuelve el resultado al programa llamador.

La declaración de la función será:

```
< tipoResultado> funcion < nombreFuncion> (lista de parametros)
    [declaraciones locales]
    Inicio
    //cuerpo de la función
    return (< expresión>)
    fin_función
```

La sentencia devolver (return, volver) se utiliza para regresar de una función, devolviendo el control del programa al llamador de la función.



5.2. Invocación de funciones

Una función puede ser llamada de la siguiente forma:

nombre_función (lista de parámetros actuales).

Donde:

- **nombre_función**, es la función a la que se llama.
- **lista de parámetros actuales**, son constantes, variables, expresiones, valores de funciones. Nombres de funciones o procedimientos.

Cada vez que se llama a una función desde el algoritmo principal, se establece automáticamente una correspondencia entre los parámetros formales y los parámetros actuales.

Debe haber exactamente el mismo número de parámetros actuales que de parámetros formales en la declaración de la función. Se presupone una correspondencia uno a uno de izquierda a derecha entre los parámetros formales y los actuales.

Una llamada a la función implica los siguientes pasos:

- A cada parámetro formal se le asigna el valor real de su correspondiente parámetro actual.
- Se ejecuta el cuerpo de acciones de la función.
- Se devuelve el valor de la función y se retorna al punto de llamada.

5.3. Parámetros formales y actuales

Parámetros formales y actuales en los subalgoritmos:

- **Parámetro formal.**

Son los parámetros que aparecen en la definición de un subalgoritmo, función o procedimiento, escritos en la cabecera de éste. Actúan como variables "molde" o "contenedor", que recibirán los valores concretos (parámetros actuales) cuando el subalgoritmo sea invocado.

Los parámetros formales sólo pueden ser variables.

- **Parámetro actual.**

También llamados **argumentos**. Se encuentran en la llamada al subalgoritmo. Son los valores, constantes, variables o expresiones que se pasan en la llamada a un subalgoritmo (función o procedimiento). Representan los datos concretos que sustituyen a los parámetros formales en el momento de la invocación.



Imprescindible

Cada vez que, un subalgoritmo es llamado, se establece una correspondencia entre los parámetros actuales o argumentos y los parámetros formales de tipo posicional. Es importante respetar la consistencia de tipos. Si no hay consistencia numérica y/o de tipo, el compilador detectará un error.

5.3.1. Parámetros. Paso por valor o por referencia

El conjunto ordenado de todos los parámetros que aparecen en su definición (parámetros formales) o en su invocación (parámetros actuales) se denomina lista de parámetros.

Cuando se invoca una función o procedimiento, cada parámetro formal toma como valor inicial el valor del correspondiente parámetro actual.

Cuando un programa llama a un subprograma, función o procedimiento, la información se comunica a través de la lista de parámetros y se establece una correspondencia automática entre los parámetros formales y actuales.

Los métodos más empleados para realizar el paso de parámetros a una función son:

- **Paso por valor** (también conocido por parámetro valor).

Los parámetros se tratan como variables locales y los valores iniciales se proporcionan copiando los valores de los correspondientes argumentos.

Los parámetros formales (locales a la función) reciben como valores iniciales los valores de los parámetros actuales y con ello se ejecutan las acciones descritas en el subprograma. Los parámetros son solo de entrada. La llamada por valor no devuelve información al programa que llama.

- **Paso por referencia** o dirección (también conocido por parámetro variable).

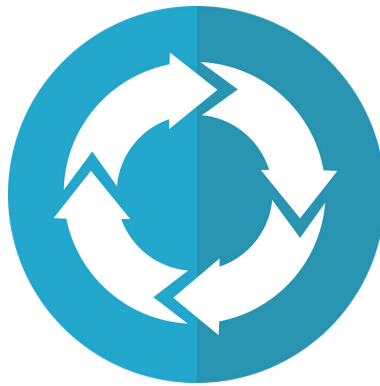
En numerosas ocasiones se requiere que ciertos parámetros sirvan como parámetros de salida, es decir, se devuelvan los resultados a la unidad o programas que llama.

Este método se denomina paso por referencia, o también llamada por dirección o por variable. En este caso, la invocación pasa como argumento la dirección de memoria de la variable, lo que permite que el subalgoritmo pueda modificar directamente su valor en el ámbito original.

En el paso por referencia, el parámetro formal se asocia a la dirección de memoria del parámetro actual. De este modo, el subprograma accede y puede modificar directamente el valor de la variable original, lo que convierte a los parámetros en elementos de entrada y salida.



6. Recursividad



Fuente: Pixabay

La recursividad es la propiedad mediante la cual un subprograma o rutina puede llamarse a sí mismo para realizar una tarea, es decir, se define en función de sí mismo.

Es una herramienta muy potente y útil en la resolución de problemas de naturaleza recursiva. Para evitar que la recursión continúe indefinidamente es preciso incluir una condición de terminación.

Ejemplo: cálculo de un número factorial

Planteamiento del problema:

Sin recursividad:

$n! = 1$ si $n = 0$

$n! = n * (n-1) * (n-2) * \dots * 1$ si $n > 0$

Recursivamente:

$n! = 1$ si $n = 0$

$n! = n * (n-1)!$ si $n > 0$

Pseudocódigo:

entero: función factorial(**E entero:** n)

//calcula recursivo del factorial

Inicio

si $n = 0$ **entonces**

devolver (1)

si_no

devolver ($n * \text{factorial}(n - 1)$)

fin_si

fin_función



7. Principales lenguajes de programación



Fuente: <https://commons.wikimedia.org/wiki/File:Prog-languages.png>

Vamos a indicar a continuación, los principales lenguajes de programación.

Puedes consultar el índice de la comunidad de programación TIOBE, que indica una medida de la popularidad de los lenguajes de programación (lo cual no significa que sean los mejores). Se basa en los principales motores de búsquedas para determinar cuáles son los principales lenguajes de programación.



+ Info

El índice de popularidad, es mantenido por la Compañía TIOBE con sede en Eindhoven, Países Bajos TIOBE significa The Importance of Being Earnest, (el título de una comedia de 1895 de Oscar Wilde).

Puedes consultarlos en cualquier momento en la web de TIOBE.

<https://www.tiobe.com/tiobe-index/>

7.1. Lenguajes de programación más destacados

Vamos a ver una descripción breve de los siguientes lenguajes:

- C.
- C++.
- C#.
- Java.



- JavaScript.
- PHP.
- Python.
- VISUAL BASIC .NET (VB.NET).
- SQL.
- PL/SQL.

7.1.1. C



Fuente:
https://commons.wikimedia.org/wiki/File:The_C_Programming_Language_logo.svg

El lenguaje C es del tipo lenguaje estructurado. Es un lenguaje de programación de propósito general que ofrece economía sintáctica, control de flujo, estructuras sencillas y un buen conjunto de operadores.

Se le suele llamar lenguaje de programación de sistemas debido a su utilidad para escribir compiladores y sistemas operativos, aunque de igual forma se puede desarrollar cualquier tipo de aplicación.



+ Info

El lenguaje C, es capaz de manejar letras como si fueran números.

El usuario es el responsable de llamar a las funciones correspondientes.



En Pascal, es posible concatenar las cadenas de caracteres con el operador "suma", pero el usuario no puede llamar a las funciones correspondientes.

```

17 string sInput;
18 int iLength, iN;
19 double dblTemp;
20 bool again = true;
21 while (again) {
22     iN = -1;
23     again = false;
24     getline(cin, sInput);
25     system("cls");
26     stringstream(sInput) >> dblTemp;
27     iLength = sInput.length();
28     if (iLength < 4) {
29         again = true;
30         continue;
31     } else if (sInput[iLength - 3] != '.') {
32         again = true;
33         continue;
34     } while (++iN < iLength) {
35         if (isdigit(sInput[iN])) {
36             continue;
37         } else if (iN == (iLength - 3)) {
38             continue;
39         }
40     }
41 }

```

Fuente: <https://pxhere.com/en/photo/767339>

El lenguaje C es un lenguaje de nivel intermedio entre los lenguajes de alto y bajo nivel, permitiendo beneficiarse de las ventajas de ambos tipos de lenguajes, y reduciendo sus inconvenientes.



+ Info

El **lenguaje C** fue creado a mediados de los años 70, por Dennis Ritchie y Brian Kernighan.

Se implementó por primera vez por Dennis Ritchie sobre un ordenador DEC PDP-11, con S.O. UNIX.

Puesto que desde la creación del lenguaje C, surgieron diferentes versiones, con diferencias palabras reservadas etc. Fue necesario unificar el lenguaje C, creándose el estándar de C, llamado ANSI-C. De esta forma en programa en C es portable de un compilador a otro y de un ordenador a otro.



Importante

ANSI-C.

Estándar que declara una serie de características, etc., que debe cumplir todo lenguaje C.

Por ello, y dado que todo programa que se desarrolle siguiendo el standard ANSI de C será fácilmente portable de un modelo de ordenador a otro modelo de ordenador.

7.1.1.1. Características del lenguaje C

Sus principales características son:

- Es un **lenguaje estructurado**, pero no por bloques, es decir no permite declarar subrutinas dentro de otras subrutinas. (como en Pascal, Ada o Modula-2).
- C permite la conversión y asignación entre diferentes tipos de datos de un modo fácil, no es rígido en la comprobación de tipos de datos.
- Es un lenguaje de propósito general. Se ha utilizado para el desarrollo de muy diversas aplicaciones: sistemas operativos, hojas de cálculo, gestores de bases de datos...
- Proporciona una gran flexibilidad de programación.
- Baja comprobación de incorrecciones, dejando bajo la responsabilidad del programador acciones que otros lenguajes realizan por sí mismo. Por ejemplo, C no comprueba que el índice de referencia de un array no sobrepase el tamaño del mismo, etc.
- Es un lenguaje portable: es independiente del *hardware* y del sistema operativo. Los programas escritos en C son fácilmente portables a otros sistemas.
- El compilador debe ser lo más pequeño y eficiente posible.
- Tiene pocas palabras reservadas y un conjunto reducido de sentencias.
- No existe anidamiento de procedimientos.
- La entrada/salida no se considera parte del lenguaje en sí, sino que se apoya a través de funciones de librería.



- Tiene gran riqueza de operadores. Prácticamente dispone de un operador para cada una de las posibles operaciones en código máquina.
- Es un lenguaje relativamente pequeño. Se puede describir en poco espacio y aprender rápidamente.

7.1.1.2. Entorno de C

C, nos ofrece un entorno de trabajo para crear, compilar y ejecutar programas en lenguaje C.

- **Preprocesador:** es un programa auxiliar, que como su nombre indica, preprocesa los fuentes en C.

Es el primer programa invocado por el compilador, que procesa otras directivas.

Las directivas comienzan con #.



Ejemplo

```
#include ejemplo.h
```

Esta directiva invocará al preprocesador que sustituirá la directiva por el contenido completo del archivo indicado. Por eso se dice que el archivo se incluye en el código fuente.

- **Archivos de cabeceras** (header file o include file): en español fichero de inclusión.

Son directivas para el procesador, archivos fuente con extensión .h, que el compilador incluye de forma automática al procesar otro archivo fuente.

Un header file contiene, normalmente, una definición de funciones, constantes, subrutinas etc. Cuando un programador desea declarar identificadores estándares en más de un archivo fuente, los coloca en un único header file. De esta forma posteriormente lo que incluirá cuando lo requiera.

Sintaxis: `#include<nombre_archivo.h>`



Normalmente, la inclusión de los header files se especifica mediante directivas `#include` al comienzo (cabecera) de otro archivo fuente.



+ Info

`#pragma once` // Evita que este archivo se incluya más de una vez

Además de las directivas existen los pragmas que son directivas del preprocesador en C/C++ que envían instrucciones especiales al compilador, y su efecto depende del compilador usado.

- **Librerías o bibliotecas:** recopilación de ficheros con rutinas, estandarizadas por un comité de la Organización Internacional para la Estandarización (ISO), que implementan operaciones comunes, como leer el teclado, escribir en la pantalla, manejar números, realizar funciones matemáticas, etc.

7.1.1.3. Variables; declaración y asignación

Cuando declaramos una variable, se indica el tipo de dato determinado seguido de su nombre. Estas variables se usarán en cálculos, consultas etc.

C permite realizar la inicialización de las variables al momento de ser declaradas.

Hay que definir las, especificando el nombre y el tipo a que pertenecen, se le puede dar o no un valor inicial. Cada tipo de variable se codifica de forma diferente, y de forma implícita, al declararlas se le asigna un espacio y lugar en memoria.

Tipos:

- **Punteros a variables:**

En C, los punteros son tipos de datos derivados (no primitivos).

Existen punteros de distintos tipos definidos por la especificación de su tipo base: `int*`, `char*`, `float*`, etc.

El operador * cumple en C dos funciones bien distintas según el contexto si es en declaración o en expresiones:

- **en la declaración:** indica que una variable es un puntero al tipo base (por ejemplo, `int *p;`)
- **en expresiones:** desreferencia el puntero (`*p`) para acceder al contenido de la dirección que almacena.



Cuando asignamos y como decíamos, el puntero debe de concordar en tipo con el valor guardado en la dirección a la que apunta, es decir, un puntero de tipo `char*` solo puede usarse de manera segura para apuntar a una dirección que contenga datos de tipo `char`.

El operador `&` será quien nos devuelva la dirección de memoria de una variable. A un puntero se le ha de asignar una dirección de memoria, el operador que nos la devolverá es `&`.

A continuación, vemos como se emplean estos operadores.

En la declaración:

Mostramos distintas maneras de proceder a una declaración y asignación de un puntero.

En dos pasos:

```
int numero, *pun; //declaración de variables
numero = 25; //asignación de la variable entera numero.
pun = &numero; //asignación de la variable puntero.
```

En un solo paso:

```
int numero = 25, *pun = &numero;
```

Cuando declaramos un puntero se ha de asignar una dirección de memoria acorde con ese tipo de punteros.

```
int a = 27;
char m = 'R';
int *i = a; // Error: se intenta asignar un valor entero a un puntero
int *i = &m; // Error: &m es de tipo puntero a char y i es puntero a int
int *i = &a; // Correcto: coinciden los tipos (puntero a int)
```



Pero existe una excepción: los arrays, por su estructura, se deben de asignar sin indicar de manera explícita el operador &:

```
int v[] = {1,2,3,4,5};
int *i = v; // Correcto: el array decae a puntero al primer elemento (int *)
int *i = &v; // Error: &v es de tipo puntero a array de 5 enteros (int (*)(5))
int *i = &v[0]; // Correcto: apunta a la dirección del primer elemento del array
```

En expresiones:

Si usamos el operador * fuera de las declaraciones, nos devolverá el contenido de la dirección de memoria apuntada, esto es, el valor. Estamos desreferenciando, es decir, con el operador * (a diferencia del mismo operador en las declaraciones) estamos indicando que queremos acceder al contenido de esa dirección de memoria.

```
int suma, numero = 25, *pun = &numero; // Declaraciones
suma = *pun + 3; // En expresiones. Desreferenciación: suma valdrá 28
```

En este último ejemplo vemos el uso del operador * en sus dos facetas: en la declaración, donde define la variable pun como puntero, y en una expresión, en la que se accede al valor almacenado en la dirección de memoria a la que apunta dicho puntero.

- **Char:** para representar un grupo de caracteres, perteneciente a un determinado código utilizado por el ordenador (normalmente el código ASCII).

Se escribe: Char identificador = 'valor';

Ejemplos:

- char letra, letra2;
- char letra='a';
- **int:** define tipos de datos enteros.

Se escribe: int nombre_variable = valor;

No es necesario que la variable tenga un valor predeterminado. Se puede definir sin asignarle ningún valor.



Si tenemos varios datos que son del mismo tipo, se pueden definir todas en la misma línea de código escribiendo un único `int`, separando el nombre de las variables por `,`. Una vez que se haya acabado de definir variables, se cierra la línea de código con `;`,

Ejemplos:

- `int edad;`
- `int edad = 24;`
- `int edad, num, contador;`
- **float:** definir datos reales (números reales con decimales).

Se escribe: `float identificador = valor;`

Ejemplos:

- `float numero1, numero2;`
- `float numero3 = 123.43;`
- `float numero3;`
- **void:** define una función que no devuelve ningún valor.

Impedir cambio de valor de una variable

Mediante el modificador `const`, podemos impedir que se modifique el valor de una variable después de inicializarla.

Se escribe: `const tipo nombre=valorinicial;`

Ejemplo: `const int num=827;`

Si el programa contiene sentencias que impliquen modificar esta variable `num`, el programa no se compilará.

7.1.1.3.1. Definir un array

Se puede definir un array (matriz o arreglo) de diferentes tipos de datos, en la sintaxis debemos indicar primero el tipo de datos, a continuación, el nombre del array y entre corchetes el tamaño máximo de elementos del mismo, terminando con "punto y coma". Si se inicializa con valores y no se indica entre corchetes ningún valor, este será el número de valores asignados.



Sintaxis: `tipodedato nombredelvector[tamaño];`

Ejemplos en C++ de arrays con nombre miarray:

- `int miarray[10];`
Contendrá un máximo de 10 elementos de tipo entero.
- `float miarray[25];`
Contendrá un máximo de 25 elementos de tipo float.
- `string miarray[400];`
Contendrá un máximo de 400 elementos de tipo string.
- `bool miarray[1000];`
Contendrá un máximo de 1000 elementos de tipo booleano.
- `char miarray[2];`
Contendrá un máximo de 2 elementos de tipo char.

Si queremos inicializarlo con valores, indicaremos un `=` y a continuación, entre llaves `{}`, cada uno de los valores entrecomillados, y separados por una coma.

Ejemplo: `int miarray[] = {41,2,4,7};`

En este ejemplo, inicializamos un array de tipo de datos enteros, con nombre `miarray` y con cuatro valores (41, 4, 4 y 7). Como no especificamos el tamaño entre los corchetes, el array tendrá como tamaño el número de elementos que hemos indicado entre las llaves, concretamente cuatro valores.

En C# los corchetes que indican el tamaño del array se indican a continuación del tipo de dato en lugar de a continuación del nombre del array, y para iniciarlo se indica "new" quedando así: `int [ ] miarray = new int[] {41,2,4,7};`



+ Info

También pueden crearse arrays multidimensionales, utilizando el operador `new`. Puedes consultar más información en la web de Microsoft.

<https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/arrays/>



7.1.1.4. Tokens de C

Son componentes sintácticos en el lenguaje C. Se dividen en 6 clases:

- Palabras claves o reservadas (keywords).
- Identificadores.
- Constantes.
- Operadores.
- Separadores.
- Comentarios.

7.1.1.4.1. Palabras claves o reservadas (keywords)

Su uso está predefinido en el propio lenguaje C, por lo que el programador no puede utilizarlas como identificadores (nombres de variables y/o de funciones).

Los keywords al ordenador que realice una tarea y tienen un especial significado para el compilador. El lenguaje C posee un número reducido de palabras reservadas, tan solo 32 palabras:

auto	double	int	struct	break	else	long	switch
case	enum	register	typedef	char	extern	return	union
const	float	short	unsigned	continue	for	signed	void
default	goto	sizeof	volatile	do	if	static	while

Palabras reservadas del Lenguaje C

7.1.1.4.2. Identificadores

Son nombres que hacen referencia a una función o a alguna variable. En ANSI C las reglas son las siguientes:

- El primer carácter de un identificador debe ser siempre una letra o un subrayado (underscore) "_".
- Los siguientes pueden ser letras, excepto "ñ" (desde a hasta z, desde A hasta Z) dígitos (del 0 al 9) o subrayado "_".
- Un identificador no puede contener la letra ñ, espacios en blanco, ni otros caracteres distintos de los citados, como por ejemplo (*,;,:-+, etc.).
- Se hace distinción entre letras mayúsculas y minúsculas.
- ANSI C permite definir identificadores de hasta 31 caracteres de longitud.



7.1.1.4.3. Constantes

Son valores que se escriben directamente en el código, y una vez compilado, no puedes modificarse.

Son tipos de datos:

- **Enteras:** números enteros con o sin signo, que estarán compuestos por los dígitos del 0 al 9, pudiendo ser positivos o negativos (9, -9).
- **Numéricas:** números reales con decimales.
- **Carácter:** un carácter, perteneciente a un determinado código utilizado por el ordenador (normalmente el código ASCII).
- **Cadena de caracteres:** conjunto de caracteres.

7.1.1.4.4. Operadores

Son cadenas de 1 o 2 caracteres, que indica al programa algo que debe hacer:

Operadores matemáticos

Aritméticos					
+	-	*	/	%	
Asignación					
=	+=	-=	*=	/=	
Incrementales					
++			--		
Asignación					
>	>=	<	<=	==	≠
	≥		≤		!=
Mayor	Mayor o igual	Menor	Menor o igual	Igual	Diferente



Lógicos

La siguiente tabla indica los operadores lógicos:

C	Descripción
&&	And, y, conjunción
	Or, o, disyunción
!	Not, no, negación

El resultado sólo puede ser un dígito que puede ser:

- 0: valor falso.
- 1: valor verdadero (cierto).

Para saber el resultado, hay que conocer las tablas de verdad:

	Not
a	!a
0	1
1	0

		Conjunción	Disyunción
a	b	a && b	a b
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1



Operadores bit a bit

Se aplica la operación lógica a cada uno de ellos.

C	Descripción
&	And bit a bit
	Or bit a bit
~	Complemento a uno o negación bit a bit
^	OR-exclusivo bit a bit

Las tablas de verdad, aplicadas bit a bit a los operandos, son:

		AND	OR	OR EXCLUSIVO
a	b	& a	a b	a ^ b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

~ COMPLEMENTO A UNO o NEGACION BIT A BIT

Se define como el valor obtenido al invertir todos los bits en la representación binaria del número (intercambiando 0 por 1 y viceversa).

~0100011 = 1011100

~1111111 = 0000000

~0000000 = 1111111

Operador condicional

Es un operador ternario, es decir, tiene tres operandos.

	Descripción
?:	Operador condicional $c ? e_1 : e_2$



Su funcionamiento es el siguiente:

Para una ejecución $c ? e_1 : e_2$ siendo c , e_1 y e_2 tres expresiones.

- Se evalúa c .
 - Si el resultado es cierto (es decir, distinto de cero).
 - » Se evalúa e_1 y éste será el resultado.
 - Si no (el resultado es falso o cero).
 - » Se evalúa e_2 y éste será el resultado.

Ejemplo:

```
Para x = 4, y = 2, z = 7:
La expresión x >= 5? 1:0 se evalúa a 0
La expresión x >= 5? y:z se evalúa a 7
La expresión x <= 5? y:z se evalúa a 2
La expresión x ? z+y:z-y se evalúa a 9
La expresión x >= 0 ? sqrt(x):0 se evalúa a 2.0 (sqrt: raíz cuadrada, prototipo en math.h)
```

Precedencia y asociatividad de los operadores

La prioridad y asociatividad de los operadores de C afectan a la agrupación y evaluación de los operandos en las expresiones.

La prioridad de un operador solo es significativa si otros operadores con una prioridad mayor o menor están presentes.

Las expresiones con operadores de mayor prioridad se evalúan primero. La prioridad también se puede describir con la palabra "enlace". Se dice que los operadores con mayor prioridad tienen un enlace más estricto.

En la tabla siguiente se resume la prioridad y asociatividad (el orden en que se evalúan los operandos) de los operadores de C, que se enumeran por orden de prioridad, de mayor a menor.

Cuando varios operadores aparecen juntos, tienen la misma prioridad y se evalúan según su asociatividad.



Prioridad y Asociatividad de los operadores

Símbolo	Tipo de operación	Asociatividad
[] () . -> ++ -- (sufijo)	Expresión	De izquierda a derecha
sizeof & * + - ~ ! ++ -- (prefijo)	Unario	De derecha a izquierda
Typecasts	Unario	De derecha a izquierda
* / %	Multiplicativo	De izquierda a derecha
+ -	Aditivo	De izquierda a derecha
<< >>	Desplazamiento bit a bit	De izquierda a derecha
< > <= >=	Relacional	De izquierda a derecha
== !=	Igualdad	De izquierda a derecha
&	AND bit a bit	De izquierda a derecha
^	OR exclusivo bit a bit	De izquierda a derecha
	OR exclusivo bit a bit	De izquierda a derecha
&&	AND lógico	De izquierda a derecha
	OR lógico	De izquierda a derecha
? :	Expresión condicional	De derecha a izquierda
=*/= %= += -= << = >>= & = ^= =	Asignación simple y compuesta	De derecha a izquierda
, (símbolo de coma)	Evaluación secuencial	De izquierda a derecha

- Los operadores se enumeran por prioridad, de mayor a menor. Si aparecen varios operadores en la misma línea o en un grupo, tienen la misma prioridad.
- Todos los operadores de asignación simples y compuestos tienen la misma prioridad.

Una expresión puede contener varios operadores con la misma prioridad.

Cuando varios operadores de este tipo aparecen en el mismo nivel en una expresión, la evaluación continua según la asociatividad del operador, de derecha a izquierda y de izquierda a derecha.

La dirección de evaluación no afecta a los resultados de las expresiones que incluyen más de un operador de multiplicación (*), adición (+) o binario bit a bit (&, | o ^) en el mismo nivel.



El orden de las operaciones no lo define el lenguaje. El compilador es libre de evaluar estas expresiones en cualquier orden, si puede garantizar un resultado coherente.

Solo los operadores de evaluación secuencial (,), AND lógico (&&), OR lógico (||), expresión condicional (?:) y llamada a función constituyen puntos de secuencia y, garantizando un orden de evaluación concreto para sus operandos.

El operador de llamada a función es el conjunto de paréntesis que siguen al identificador de función. Está garantizado que el operador de evaluación secuencial (,) evalúa sus operandos de izquierda a derecha. (El operador de coma en una llamada a función no es igual que el operador de evaluación secuencial y no proporciona esta garantía).

Los operadores lógicos también garantizan la evaluación de sus operandos de izquierda a derecha. Sin embargo, evalúan el número más pequeño de operandos necesarios para determinar el resultado de la expresión. Esto se denomina evaluación de "cortocircuito". Por tanto, es posible que algunos operandos de la expresión no se evalúen.

Por ejemplo, en la expresión `x && y++`

Se evalúa el segundo operando, `y++`, solo si `x` es true (distinto de cero).



+Info

Información obtenida de:

<https://docs.microsoft.com/es-es/cpp/c-language/precedence-and-order-of-evaluation?view=vs-2019>

7.1.1.4.5. Separadores

Son las constantes de caracteres de barra invertida.

Se usan para introducir caracteres que es imposible introducir por el teclado (tales como retorno de carro, etc.).

Estas constantes son proporcionadas por C para que sea posible introducir dichos caracteres como constantes en los programas en los cuales sea necesario.



Estas constantes de caracteres de barra invertida son:

Código	Significado
<code>\b</code>	Retroceso
<code>\f</code>	Alimentación de hoja
<code>\n</code>	Nueva línea
<code>\r</code>	Retorno de carro
<code>\t</code>	Tabulador horizontal
<code>\"</code>	Doble comilla
<code>\'</code>	Simple comilla
<code>\0</code>	Nulo
<code>\\</code>	Barra invertida
<code>\v</code>	Tabulador vertical
<code>\a</code>	Alerta
<code>\o</code>	Constante Octal
<code>\x</code>	Constante Hexadecimal

El uso de las constantes de barra invertida es igual que el de cualquier otro carácter, así, si `ch` es una variable de tipo `char`, podemos hacer: `ch=' '`, o `ch='x20'` (carácter espacio), etc., de igual forma que realizaríamos con cualquier otra constante de carácter.

Además, las constantes de barra invertida pueden usarse en el interior de constantes de cadena como un carácter más, por ello, podemos poner escribir la constante de cadena: "Esto es una línea ".

7.1.1.4.6. Comentarios

Son cadenas de caracteres que introducimos para indicar qué es lo que hace determinada función o qué proceso realiza determinada sentencia. Se utilizan con fines de documentación.

Un comentario se especifica de la siguiente forma:

```
/* Este es un comentario */
```




Todo lo que se escriba entre estos dos elementos, es decir, `/*.....*/`, será ignorado por el compilador y no formará parte del código ejecutable del programa final.



+ Info

El lenguaje ANSI C permite también otro tipo de comentarios, tomado del C++. Todo lo que va en cualquier línea del código detrás de la doble barra (`//`) y hasta el final de la línea, se considera como un comentario y es ignorado por el compilador.

7.1.1.5. Agrupación de Tokens

Los Tokens se agrupan en **sentencias** que pueden ser:

- Simples: terminan en `;`
- Compuestas: compuestas de varias sentencias simples, indicadas entre paréntesis `{ }`



Atención

Sentencias ("Statements").

Especifican y controlan el flujo de ejecución del programa. Si no existen sentencias específicas de selección o salto, el programa se ejecuta de forma secuencial en el mismo orden en que se ha escrito.

7.1.1.6. Librerías

Las librerías ANSI que se incluyen con todos los compiladores están escritas en C o en ensamblador, y por lo tanto no son "imprescindibles" para escribir programas en C.



Librerías ANSI C:

- assert.
- ctype.
- errno.
- float.
- limist.
- locale.
- math.
- setjmp.
- signal.
- stdarg.
- stddef.
- stdio.
- stdlib: (stdlib.h: std-lib: standard library o biblioteca estándar). Es el archivo de cabecera de la biblioteca estándar de propósito general del lenguaje de programación C. Contiene los prototipos de funciones de C para gestión de memoria dinámica, control de procesos y otras. Es compatible con C++ donde se conoce como cstdlib.

`gotoxy(argumento1,argumento2);`

En el primer argumento se especifica el número de columna (1 - 80) y en el segundo argumento el número de renglón (1 - 24). Esta función se encuentra dentro de la librería.

- string.
- time.

Librerías BGI (Borland Graphics Interface)

Librerías no estándar ofrecidas por Borland, al no ser estándar los programas que hagan uso de ellas no serán necesariamente portables a otras plataformas ni a otros compiladores.



Notas

Algunas descripciones de funciones, estructuras y macros han sido extraídas de la ayuda de los compiladores de Borland y del libro: "C How to Program" de H.M. DEITEL & P.J. DEITEL.

- **Librería conio Borland ® C.**

La librería conio.h sirve para cambiar el fondo de pantalla, dar color al texto, etc. y, cuando termine de realizar todas las operaciones limpiar la pantalla (getch(), o, getche())

"conio.h es un encabezado de archivo C utiliza en los compiladores antiguos de MS-DOS para crear interfaces de usuario de texto.

No se describe en el libro Lenguaje de Programación C, y no es parte de la biblioteca C estándar, ISO C ni es requerido por POSIX.

Este encabezado declara varias funciones útiles para realizar la colección de "consola de entrada y salida" de un programa.

La mayoría de los compiladores de C que se dirigen a UNIX y Linux no tienen este encabezado y no proporcionan las funciones de biblioteca concomitantes.

Algunos sistemas embebidos están utilizando una biblioteca Conio compatible.

Las funciones de la biblioteca conio.h declarado pueden ligeramente de compilador.

Veamos una de las funciones destacadas:

- **Funcion: gotoxy().**

Sintaxis: void gotoxy(int x, int y);

Descripción: Mueve el cursor de la ventana de texto a la posición según las coordenadas especificadas por los argumentos x e y. Si las coordenadas no son válidas entonces la llamada a la función gotoxy es ignorada. Los argumentos no pueden ser 0.

- **Librería graphics.h.**

Con esta librería, podemos incluir los gráficos BGI en nuestras aplicaciones.

7.1.1.7. Estructura de un programa en C

Todo programa de C consta, básicamente, de un conjunto de funciones, y una función llamada main, la cual es la primera que se ejecuta al comenzar el programa, llamándose desde ella al resto de funciones que compongan nuestro programa.



Ejemplo:

La expresión siguiente es válida en C:

```
float a; /*Declaro una variable para números reales*/  
int b; /*Declaro otra variable para números enteros*/  
b=a; /*Asigno a la variable para entera el número real*/
```

Función main

Es la primera función que se ejecuta, representa el cuerpo principal del programa.

Sintaxis: (tipo) main () {cuerpo del programa}.

Cada declaración de variables o instrucción debe terminar con punto y coma.

```
cabecera  
    declaraciones globales;  
main()  
{  
    declaración variables locales;  
    flujo de sentencias  
}  
funcion_1()  
{  
    declaración variables locales;  
    flujo de sentencias  
}  
.  
.  
funcion_n()  
{  
    declaración variables locales;  
    flujo de sentencias  
}
```



7.1.1.8. Funciones de C

Se pueden diferenciar entre las funciones proporcionadas por el lenguaje C nativas (de biblioteca o integradas) y las realizadas por usuario.

En general, las funciones son rutinas que sirven para descomponer un programa en varios módulos pequeños.

Poseen las siguientes características:

- Pueden estar en el mismo fichero que el programa principal o compilarse por separado.
- Pueden ser llamadas por el programa principal o por otra función.
- Pueden llevar argumentos (opcional).
- Pueden devolver un valor (opcional).
- Pueden poseer sus propias variables.
- El C no se distingue entre procedimiento y función.
- No permite anidamiento de funciones.

Una función tiene la siguiente sintaxis:

```
Tipo Nombre_funcion (lista_argumentos)
    Declaración
    {
        variables locales
        instrucciones
        ...
    }
```

La función nativa printf()

Si hablamos de funciones nativas, una de las más usadas sino la más usada es la función **printf()**. Esta función viene por defecto con la librería `<stdio.h>`, librería necesaria si queremos acceder a funciones estándar de entrada y salida.

Dada su importancia nos detenemos explicamos un poco su modus operandi.



```
#include<stdio.h>

int main() {
    int sobres_levadura = 1;
    int vasos_agua = 2;
    float kilos_harina = 1.25;
    printf("Necesitará %.2f kilos de harina, %d sobres de levadura y %d vasos de agua y .\n", kilos_harina, sobres_levadura, vasos_agua);
    return 0;
}
```

La función **printf()** formateará los valores de *kilos_harina*, *sobres_levadura* y *vasos_agua* según los especificadores `%.2f` y `%d`, respectivamente, y los reemplazará en la cadena de formato antes de imprimir el mensaje. Vemos en el caso del especificador del número real se agrega un `".2"` que indicará que se requieren 2 decimales.

Así pues, en la función `printf` usaremos literales y especificadores que serán sustituidos en la ejecución por el contenido de las variables invocadas.

A continuación, vemos una lista de especificadores de formato más comunes:

- `%d` o `%i`, números enteros con signo.
- `%u`, números enteros sin signo.
- `%f`, números reales.
- `%c`, caracteres.
- `%s`, cadenas de caracteres.

7.1.1.9. Memoria dinámica: fugas, dobles liberaciones y el infierno del HEAP

En C no hay recolector. Si `malloc` devuelve memoria, `free` debe liberarla. Si no queda claro quién es el dueño, aparecen las deudas técnicas que se pagan en incidencias a las 3 AM.

Tres memorias, tres vidas

- **Stack:** variables locales. Se liberan solas al salir de la función. Rápido, pero limitado (un stack de 8 MB puede saturar con arrays grandes).
- **Heap:** `malloc`/`calloc`/`realloc`. Manual. Lento, pero flexible. Aquí viven los leaks.
- **Estática/Global:** vive durante todo el proceso. Útil para configuración, pero peligrosa en concurrencia.



Ownership: la pregunta que nadie se hace

¿Quién reserva? ¿Quién libera? ¿Y si hay un error a mitad de camino? Las funciones deberían documentar esto, pero en legado no lo hacen. Un patrón seguro: la función que reserva es la que libera, o devuelve memoria con un contrato claro (y un free asociado).

Los tres errores que se pagan caro

- **Memory leak:** reservas y no liberas. En un servicio que vive semanas, termina en OOM killer o reinicio forzado.
- **Double free:** liberas dos veces. Corrompe el heap. Los síntomas aparecen después, en lugares aleatorios.
- **Use-after-free:** usas memoria liberada. Puede "funcionar" durante días hasta que el sistema reasigna ese bloque y todo explota.

Herramientas reales (nunca se mencionan pero importan)

valgrind --leak-check=full en Linux, Dr. Memory en Windows. Pero en producción no siempre están disponibles. Lo que sí tienes es el contador de memoria del proceso. Si crece sin parar, hay un leak.

7.1.1.10. Interoperabilidad: cuando C se encuentra con Java/.NET (JNI/P/Invoke)

C suele aparecer como capa nativa llamada desde plataformas gestionadas. Y en esa frontera está el 80% de los problemas "imposibles de reproducir".

Entrada/Salida: la frontera del sistema

C está cerca de los ficheros, sockets y dispositivos. Los errores clásicos:

- No comprobar retornos: fread puede leer menos bytes sin ser error.
- errno no se resetea solo. Hay que leerlo inmediatamente tras el fallo.
- Rutas relativas en servicios: el directorio de trabajo no es el que crees.

JNI (Java) y P/Invoke (.NET): el salto mortal

No se trata de "llamar a C", sino de que el contrato coincida al milímetro:

- **Tamaños:** int en C puede ser 4 bytes, pero int en .NET siempre es 4. long es donde más problemas hay (4 bytes en Windows x64, 8 en Linux).



- **Cadenas:** Java usa UTF-16 modificada; .NET UTF-16; C usa UTF-8 o código página local. Convertir mal corrompe datos.
- **Alineación:** un struct en C puede tener padding entre campos. Si no usas StructLayout en .NET o `__attribute__((packed))` en ambos lados, el marshalling lee basura.
- **Ownership:** si C devuelve un `char*`, ¿quién lo libera? Si .NET pasa un buffer, ¿C puede escribir más allá del tamaño?

7.1.2. C++



Fuente:

https://upload.wikimedia.org/wikipedia/commons/1/18/ISO_C%2B%2B_Logo.svg

Basado en el lenguaje de programación C, utiliza mecanismos que permiten la manipulación de objetos. En ese sentido, desde el punto de vista de los lenguajes orientados a objetos, el C++ es un lenguaje híbrido.

Posteriormente, se añadieron facilidades de programación genérica, que se sumaron a los paradigmas de programación estructurada y programación orientada a objetos. Por esto se suele decir que el C++ es un lenguaje de programación multiparadigma.

7.1.2.1. Características del lenguaje C++

Las características principales del lenguaje de programación C++ son las siguientes:

- Sintaxis heredada del lenguaje C.
- Tiene un estándar ISO, conocido como ANSI-C++.
- Lenguaje fuertemente tipado. El programador debe saber cómo hacer y declarar el código para que funcione.
- Programación orientada a objetos.



- Abstracción.
- Encapsulado.
- Herencia.
- Polimorfismo.
- Sobrecarga de operadores.
- Soporta expresiones Lambda, también llamadas funciones anónimas.
- Control de excepciones.
- Soporte multihilo.
- Compatibilidad de C con C++. Un compilador de C++ puede compilar código escrito en C, o usar librerías de C con poca modificación de código.
- Uso de punteros.
- Es portable. Tiene un gran número de compiladores en diferentes plataformas y sistemas operativos.
- Eficiencia con el *hardware*.
- Es complejo.

7.1.2.2. Sentencias en C++. Clasificación

En C++, las sentencias se denominan Statements, y especifican y controlan el flujo de ejecución del programa.

Si no existen sentencias específicas de selección o salto, el programa se ejecuta de forma secuencial en el mismo orden en que se ha escrito el código fuente (podríamos considerarlo orden "natural" de ejecución).

Una sentencia consta de palabras clave o reservadas, expresiones, declaraciones, o llamadas a funciones. Es una secuencia de operadores; operandos; elementos de puntuación y palabras clave, que especifican la realización de un proceso. Tiene sentido computacional en sí misma, por lo que puede producir un resultado.

Ejemplo:

```
extern x; // No produce un valor
y = 22; // Produce un valor
z = i++; // Valor + efectos laterales
```



Cualquier expresión finalizada en un punto y coma ; forma una sentencia < **expresión** > ;

C++ ejecuta las sentencias evaluando la expresión. Todos los efectos colaterales de la evaluación son tenidos en cuenta antes de ejecutar la próxima sentencia.

La mayoría de sentencias C++ son asignaciones o llamadas a funciones.

Hay infinitas sentencias distintas, e innumerables criterios para su clasificación. Existe una clasificación del Estándar, que distingue las siguientes clases de sentencia:

- **De etiqueta.**

Hay tres clases:

- Las etiquetas directas.
- Las sentencias CASE.
- Las default: se utilizan en conjunción con las sentencias switch.

- **De expresión.**

Podríamos decir que son las que no pertenecen a ninguno de los otros grupos y que, en la práctica, son las más abundantes. Generalmente son asignaciones o invocaciones de funciones.

Ejemplo:

```
pint = &x;  
foo(c);
```

- **Compuestas.** (también denominadas bloques)

Se utilizan en aquellas situaciones en que la sintaxis espera una sentencia, pero se necesita usar varias.

Ejemplo:

```
if (first) ++x;  
else{  
  --x; z=a;  
}
```



En caso de cumplirse la condición hay que incrementar x, lo que puede hacerse en una sola sentencia, pero si la condición resulta falsa, se precisan dos operaciones.

Para que se comporten como una sola frente a else, se recurre a englobarlas en un bloque entre llaves { ... }.

Las sentencias dentro del bloque se comportan como una sola y constituyen un ámbito léxico.

Los identificadores definidos en su interior eclipsan a los exteriores y las variables automáticas creadas en él son destruidas al salir del ámbito.

- **De selección.**

Las sentencias de selección o de control de flujo, pueden decidir entre varios cursos de acción distintos en función de ciertos valores.

Existen dos tipos de estas sentencias de selección: if...else y switch

Ejemplo:

```
switch ( foo() ) {  
    case 0: case 1:  
        if (first) break;  
        else ++x;  
    break;  
    case 2:  
        ....  
    break;  
    default:  
        ....  
    break;  
}
```

- **De iteración.**

Las sentencias de iteración permiten repetir un conjunto de sentencias ejecutando un bucle.

En C++ existen tres formas de iteraciones:

- Los bucles while.
- do...while.
- for.



Ejemplo:

```
while (first) {  
    ...  
    for (int x = 1; x < y; ++x) {  
        ...  
        do { // begin second  
            ...  
            ...  
        } while (second); // end second  
        ...  
    } // end for  
    ...  
} // end first
```

- **De salto.**

Las sentencias de salto permiten transferir el control del programa de forma incondicional. Existen cuatro de estas sentencias:

- break.
- continue.
- goto.
- return.

Ejemplo:

```
while(foo){  
    start:  
    if(some)break;  
    ...  
    if(first)goto start;  
    ...  
    if(second) continue;  
    ...  
    if(!any) return;  
}
```



En ocasiones es posible utilizar el mecanismo de excepciones C++ como mecanismo de salto multinivel.

- **De declaración:**

Este tipo de sentencias introducen uno o más identificadores en un bloque.

Ejemplo:

```
void foo (int x, y, z){  
    float f = 3.14;  
    char c;  
    int x;  
    ...  
}
```

- **Bloques de intento.**

Estas sentencias deben estar seguidas de una sentencia **catch** y tienen la forma:

```
try {  
    int resultado = 10 / 0;  
} catch (const std::exception& e) {  
    std::cerr << "Error: " << e.what() << std::endl;  
}
```

Son utilizadas por el mecanismo de excepciones C++ y han sido expuestas con detalle en el capítulo correspondiente.

Clases Adicionales

Por sus características especiales consideramos estas como clases adicionales:

- **Sentencias de preproceso.**

Como se ha indicado, constituyen un tipo muy especial, tanto en su sintaxis como en su comportamiento. No representan una computación en tiempo de ejecución (runtime), sino de compilación, ya que su efecto es realizar modificaciones sobre el código fuente.

Sintaxis: no requieren el punto y coma de terminación y comienzan siempre con el símbolo #.



- **Sentencias ensamblador.**

Para poder escribir directamente instrucciones en lenguaje ensamblador junto con el resto del código fuente podemos usar la palabra clave específica: `asm`.

`asm` indica que la cadena literal que sigue será incluida en el código objeto en la posición indicada.

Sintaxis: depende del compilador.

Puesto que la sintaxis varía según el compilador, debemos evitar este tipo de sentencias para facilitar la portabilidad del código.

7.1.2.3. Caso especial: la sentencia nula

Consiste en un punto y coma (;) aislado.

Una sentencia nula no hace nada, pero puede ser necesaria en situaciones en que la sintaxis del lenguaje espere una sentencia, pero nuestro programa no necesita hacer nada.

La sentencia nula (; sola) es útil en bucles donde toda la lógica está en la condición: `while (rs.next());` ; procesa todo el `ResultSet`, pero es confusa. La norma del CTTI es prohibirla, requiriendo cuerpo vacío con comentario: `while (rs.next()) { /* procesado en el DAO */ }`.

Ejemplo de seguridad en bucle:

```
// PROCESO: Buscar expediente por NIF en archivo binario
for (int i = 0; i < totalRegistros; ++i) {
    if (registros[i].nif == nifBuscar) {
        resultado = registros[i];
        break; // Salida inmediata, eficiente
    }
}
// Sin break: recorre todo → O(n) innecesario
```

Existe un tipo especial de sentencia, las directivas de preproceso en las que el punto y coma puede omitirse. Este tipo de sentencias no ejecuta una computación de tiempo de ejecución sino de compilación (realizan modificaciones sobre el fuente).

Entre las que sí ejecutan una computación en runtime también existe una, las etiquetas directas, que no terminan en punto y coma sino en dos puntos :



En las sentencias directivas de preproceso, en las que el punto y coma se puede omitir, no se ejecuta una computación de tiempo de ejecución sino de compilación (realizan modificaciones sobre el fuente).

También, las etiquetas directas, que sí ejecutan una computación en runtime, no terminan en punto y coma sino en dos puntos.

7.1.2.4. Mecanismo de excepciones

Una excepción es un error.

Al ejecutar un programa, puede ocurrir una excepción que lo interrumpa de forma inesperada, por un argumento inválido para un cálculo matemático, una mala entrada por parte del usuario, un mal funcionamiento en el hardware, etc.

Para evitarlo, es muy importante que el programador escriba, los algoritmos necesarios, para evitar a toda costa que se produzca una excepción.

C++ proporciona un mecanismo, para que, de una forma directa y fácil de ver, tanto para el programador, como para los revisores del código en el manejo de excepciones. Consiste en el mecanismo:

- try.
- throw.
- catch.

La lógica del mecanismo es:

- Dentro de un bloque try se pretende evaluar una o más expresiones y si dentro de dicho bloque se produce "un algo", que no se espera se lanza por medio de throw una excepción, la misma que deberá ser capturada por un catch específico.
- Puesto que desde un bloque try pueden ser lanzados diferentes tipos de errores de excepción, es que puede haber más de un catch para capturar a cada uno de los mismos.
- Si desde un try se lanza una excepción y no existe el mecanismo catch para tratar dicha excepción, el programa se interrumpirá abruptamente después de haber pasado por todos los catches que se hayan definido y de no haber encontrado el adecuado.
- Los tipos de excepciones lanzados pueden ser de un tipo primitivo tal como: int, float, char, etc. aunque normalmente las excepciones son lanzadas por alguna clase escrita por el usuario o por una clase de las que vienen incluidas con el compilador.

C++ diferencia `std::exception` (lógica) y `std::runtime_error` (condiciones irreversibles). No hay excepciones checked; cualquier función puede lanzar. Esto exige documentar excepciones en comentarios:

```
/** @throws std::invalid_argument si NIF mal formado */
```



El catch (...) captura todo, pero pierde información. Es mejor catch (const std::exception& e) por referencia constante para evitar slicing. En integración con Java (JNI) o C# (P/Invoke), las excepciones C++ no cruzan fronteras; se traducen a int de error o se pierden, causando crashes silenciosos. La solución es catch en la capa de interoperabilidad y devolver código de error.

Nota de interoperabilidad: En la DLL de validación del DNle, usamos noexcept (C++11) para garantizar que no lanza excepciones, pues el contenedor Java espera comportamiento C puro. Si lanza, la JVM crashea irremediablemente.

7.1.2.5. Estructura de un programa en C++

C++ es un lenguaje de programación orientado a objetos híbrido.

Esto quiere decir que permite realizar programas estructurados sin la orientación a objetos y programas orientados a objetos.

Para ilustrarlo vamos a ver el ejemplo de "Bienvenido a masterD". Lo vamos a implementar utilizando programación estructurada y, a continuación, con programación orientada a objetos.

Usando programación estructurada

```
#include <iostream>
using namespace std;
int main(void)
{
    cout << "Bienvenido a masterD";
}
```

Vamos a comentar el código para entenderlo bien:

- **#include**

Esta línea es una directiva del preprocesador.

Las líneas de directivas del preprocesador empiezan con el símbolo #.

"#include" indica al preprocesador que en este programa se debe incluir la librería *iostream* (librería de entrada/salida que contiene la instrucción *cout* que usamos más adelante para imprimir un mensaje en pantalla).



- **using**

Vamos a ver diferentes usos de Using, incluyendo el del ejemplo.

- **Para importar namespace** (espacios de nombre)

Un espacio de nombres es un conjunto de nombres de recursos en el cual todos los nombres son únicos. Se utiliza para crear un bloque, y así todas las funciones que lo forman, están asociadas a ese namespace.

Todos los elementos de la biblioteca estándar de C++ se declaran dentro de un espacio de nombres llamado std.

"using namespace std" indica que vamos a usar este espacio de nombres.

- **Para importar miembros estáticos**

```
using static nombre_clase;
```

Esta característica se agregó en la versión 6.0 de C#

Si el programa que estamos desarrollando, en nuestro código o en frame-work, usa muchos miembros estáticos (métodos, campos y propiedades), resulta muy útil el uso de using static.

(Como ejemplo de clases estáticas tenemos: System.IO.File, System.Console y System.Math)

- **Para crear un alias**

No suele suceder, pero en alguna ocasión, coincide que dos o más tipos en diferentes ensamblados tienen el mismo, y si se usan ambos en una clase, se produce una colisión de nombres y el compilador no sabe que tipo debe elegir.

Este problema lo evitamos creando un alias para un tipo, mediante la palabra using seguida del alias, el símbolo igual y el nombre completo de la clase.

Ejemplo: using mialias = System.Math;

- **Para clases que implementan**

Si una clase implementa la interfaz IDisposable la utilización de using asegura que tendrá el método Dispose, utilizado para liberar recursos no administrados (como una conexión de red o con base de datos, un archivo, etc.)

Ejemplo:

```
using(System.IO.StreamReader lector = System.IO.File.OpenText("archivo.text"))
```



- **Para declaraciones using**

Con esta característica, de C# 8.0 (2019), se puede declarar una variable de forma que el recurso será liberado al final del alcance donde se declaró.

Para ello se indica la palabra `using` delante de la variable que se declara.

Ejemplo de código:

```
{  
    using (SqlConnection connection = new SqlConnection()))  
    {  
        // se realizan las operaciones de la conexión  
    } // se libera aquí la conexión  
}
```

La siguiente sintaxis es equivalente a la anterior:

```
{  
    using SqlConnection connection = new SqlConnection();  
    // se realizan las operaciones de la conexión  
} // se libera aquí la conexión
```

Así, utilizando `using`, nos aseguramos de que al acabar el bloque de código indicado entre sus llaves, se llama al método `Dispose()` del objeto con el que se ha declarado su bloque inicial, y es este método `Dispose()` el que se asegura de que se liberen los recursos de la conexión de forma correcta, y se cierre la conexión. Esto se realiza, aunque haya un error. La aplicación no deja recursos mal liberados y el programador no debe preocuparse de liberarlos ni de que una conexión este o no abierta antes de cerrarla.

Además de para conexiones, se puede utilizar `using` para objetos de GDI+, recursos transaccionales, etc.

- **namespace std;**

Un espacio de nombres es un conjunto de nombres de recursos en el cual todos los nombres son únicos.



Todos los elementos de la biblioteca estándar de C++ se declaran dentro de un espacio de nombres llamado `std`.

"`using namespace std`" indica que vamos a usar este espacio de nombres.

- `int main (void)`

Un programa en C++ estructurado (no orientado a objetos) está formado por una función *main* (función principal) y, opcionalmente, por otras funciones.

El programa siempre comienza por la función *main*. Todo programa escrito en C++ debe contener una función *main*.

- `cout << "Bienvenido a masterD";`

Imprime "Bienvenido a masterD" (sin comillas) por pantalla.

Las instrucciones acaban con punto y coma.

Usando programación orientada a objetos

Veamos el mismo ejemplo utilizando programación orientada a objetos:

```
class BienvenidoamasterD
{
    public:
    void ImprimirBienvenidoamasterD ()
    {
        std::cout << "Bienvenido a masterD";
    }
};
```

Hemos creado una clase " BienvenidoamasterD " con una función pública `ImprimirBienvenidoamasterD ()`.

Al ser pública, otras funciones fuera de esta clase pueden llamarla directamente. Como vimos en el ejemplo anterior, `cout` imprime la frase por pantalla. Esta vez hemos usado el espacio de nombres directamente con `cout`.



Para finalizar, debemos crear una instancia de la clase y llamar al método `ImprimirBienvenidoamasterD()`.

```
BienvenidoamasterD miObjetoHola;  
miObjetoHola.ImprimirBienvenidoamasterD();
```

Este código debe ir en la función `main()` para poder ejecutarse.

7.1.3. C#



Fuente:
https://commons.wikimedia.org/wiki/File:Csharp_Logo.png

También llamado C Sharp, es un lenguaje de programación orientado a objetos desarrollado y estandarizado por Microsoft como parte de su plataforma .NET.

Su sintaxis básica deriva de C/C++ y utiliza el modelo de objetos de la plataforma .NET, similar al de Java, aunque incluye mejoras derivadas de otros lenguajes.

C# es un lenguaje moderno y versátil que ofrece múltiples características para facilitar el desarrollo de software. Una de las más destacadas es el uso de tipos opcionales (nullable types), que se indica añadiendo un `?` al tipo de dato, permitiendo que valores como `int` puedan ser `null`.

```
int? numero = null; // número es opcional, puede ser null
```

Esto es útil en casos donde los datos no siempre están disponibles, como en bases de datos. Además, cuenta con el operador de coalescencia nula (`??`), que asigna un valor predeterminado si una variable es `null`, y el operador condicional nulo (`?.`), que permite acceder de forma segura a propiedades de objetos nulos sin lanzar excepciones.



```
string nombre = null;

string mensaje = nombre ?? "Usuario anónimo"; // Si nombre es null, asigna
"Usuario anónimo"

Persona persona = null;

string nombre = persona?.Nombre; // No lanza excepción, nombre será null
```

C# también incluye características modernas como las expresiones switch, que permiten evaluar patrones de forma concisa y clara, y la inferencia de tipos con var, que permite al compilador deducir el tipo de la variable. Otra funcionalidad destacada son las funciones locales, que pueden definirse dentro de otras funciones, mejorando la encapsulación y claridad del código. Además, los records, introducidos en C# 9.0, son tipos inmutables diseñados específicamente para manejar datos estructurados.

En cuanto a la programación asíncrona, C# tiene soporte nativo mediante las palabras clave **async** y **await**, facilitando el manejo de tareas no bloqueantes. También cuenta con características avanzadas como la sintaxis para trabajar con rangos y expresiones de índice, que permite seleccionar subconjuntos de datos de forma más intuitiva, y las expresiones lambda, que ofrecen una manera compacta de definir funciones anónimas.

Por último, C# simplifica el trabajo con colecciones a través de inicializadores que permiten declarar e inicializar listas o arrays directamente en una sola línea. Estas características, junto con su fuerte orientación a objetos y soporte para programación funcional, hacen de C# una herramienta potente para desarrollar aplicaciones modernas y robustas.

7.1.3.1. Estructura, Tipos y Operadores

C# nació con una filosofía de claridad que lo hace idóneo para técnicos auxiliares que migran desde entornos de script. El namespace organiza código de forma lógica, reflejando la estructura orgánica: namespace SEPE.Gestion.RRHH.Validacion. El using evita nombres completos (System.DateTime vs DateTime), pero abusar de using static System.Math reduce legibilidad en equipo.

La inferencia con var es potente: var lista = new List<Empleado>(); previene repeticiones. Sin embargo, en var resultado = Calcular(); sin nombre método claro, el tipo se oculta. La guía de estilo del INTEF limita var a inicializaciones con new o Create(). Para datos públicos, la claridad prima sobre la brevedad.

Los operadores nulos (??, ?.) son revolucionarios en integración con sistemas legados que devuelven null. string nombre = usuario?.Nombre ?? "DESCONOCIDO"; evita NullReferenceException sin verbose if. En una API del Sistema de Identidad del Ciudadano, esto redujo los errores 500 en logs en un 85%.



Tabla de equivalencia clave Java-C#:

Concepto	Java	C#	Implicación en código público
Constante	final	const / readonly	const es compile-time, readonly runtime
Casting	(int)valor	(int)valor	En C#, as devuelve null si falla
Comprobación nulo	if (x == null)	if (x is null)	is null es más legible para técnicos junior
Concatenación	+	(\$"{var}")	Interpolación evita errores de formato

7.1.3.2. Arrays, Colecciones y Métodos con Parámetros Opcionales

En C#, los arrays son objetos con métodos: `int[] codigos = new int[50]; codigos.Length` (no `length`). La inicialización `int[] {1,2,3}` es limpia. Para colecciones dinámicas, `List<T>` reemplaza `ArrayList`, y `Dictionary<string, object>` a `HashMap`. La ventaja es que C# mantiene tipado fuerte sin casting: `List<Empleado>` lista no admite objetos no `Empleado`.

Los métodos con argumentos opcionales y por defecto simplifican mantenimiento de APIs públicas. `public void EnviarNotificacion(string email, bool urgente = false, string prioridad = "normal")` permite añadir parámetros sin romper consumidores antiguos. Esto es crucial en el ENS, donde las interfaces deben ser estables. En Java, se requiere sobrecarga; en C#, una sola firma con múltiples valores por defecto.

El paso por referencia (`ref`) y salida (`out`) permite múltiples retornos sin crear objetos `Result`. `public bool TryParseNIF(string texto, out string nif, out string error)` devuelve éxito y valores. Es más eficiente que devolver null o lanzar excepciones en validaciones masivas. He comparado: procesar 100.000 NIFs con `out` es 20% más rápido que con tuplas.

Ejemplo de API pública robusta:

```
public ValidacionService(ILogger logger)
{
    _logger = logger ?? throw new ArgumentNullException(nameof(logger));
}

public bool EsValido(string nif, out string mensajeError)
```



```
{
    mensajeError = null;
    if (string.IsNullOrEmpty(nif))
    {
        mensajeError = "NIF obligatorio según Art. 22 ENS";
        _logger.Warning("Intento de validación vacía");
        return false;
    }
    // Lógica de validación...
    return true;
}

}

public class ValidacionService
{
    // readonly = inicializable solo en constructor
    private readonly ILogger _logger;
    public ValidacionService(ILogger logger)
    {
        _logger = logger ?? throw new ArgumentNullException(nameof(logger));
    }
    public bool EsValido(string nif, out string mensajeError)
    {
        mensajeError = null;
        if (string.IsNullOrEmpty(nif))
        {
            mensajeError = "NIF obligatorio según Art. 22 ENS";
            _logger.Warning("Intento de validación vacía");
            return false;
        }
        // Lógica de validación...
        return true;
    }
}
```



7.1.3.3. Clases, Modificadores y Patrón IDisposable

La jerarquía de acceso en C# (private, protected, internal, public) incluye internal, que restringe al ensamblado actual. En proyectos de la AGE con múltiples DLLs, internal oculta implementación de servicios internos, exponiendo solo public interfaces. El modificador sealed evita herencia, útil en validadores de cálculo que no deben alterarse.

Modificadores de acceso clave:

- **al:** Usa este modificador para ocultar clases que solo deben verse dentro de tu proyecto. En la AGE, con múltiples DLLs, internal class ValidadorSSFF garantiza que solo el módulo de nómina pueda acceder al validador, evitando dependencias no deseadas desde el portal público.
- **sealed:** Bloquea la herencia en clases críticas. Un sealed class ValidadorCalculoIRPF impide que alguien cree una versión modificada no auditada, protegiendo algoritmos aprobados por Intervención General.

Patrón IDisposable: cierra lo que abras

IDisposable es el equivalente a AutoCloseable de Java, pero con sintaxis using. using (var conn = new SqlConnection(cadena)) asegura la liberación incluso si hay una salida anticipada.

Los recursos (conexiones, archivos, cachés) deben liberarse siempre. Usa using:

```
// ✅ BIEN: se cierra automáticamente, aunque haya return o error
using (var conn = new SqlConnection(cadena)) {
    conn.Open();
    // ... operaciones
}
```

¿Qué pasa si no usas using? En el ServicioPago de Hacienda, un técnico olvidó el using en un bucle de 10.000 registros. El pool de conexiones se agotó en 5 minutos y todo el portal cayó. La solución fue reiniciar el servidor en plena jornada. Coste: 2 horas de servicio caído + auditoría del CCN.



Excepción crítica: MemoryCache sin Dispose

En el sistema REEST de la Seguridad Social, no llamar a `Dispose()` en `MemoryCache` generó una fuga de 2GB/día. La solución fue implementar `IDisposable`:

```
// ✗ MAL: fuga silenciosa
var cache = new MemoryCache(new MemoryCacheOptions());
cache.Set("sesion_" + userId, datos); // Al cerrar sesión, no se limpiaba

// ✓ BIEN: implementa IDisposable en tu servicio
public class SesionService : IDisposable {
    private readonly MemoryCache _cache;
    public void Dispose() {
        _cache?.Dispose(); // ¡Libera la memoria!
    }
}

// Uso en controller
using (var sesionService = new SesionService()) {
    // ... trabajo con sesiones
} // Aquí se llama automáticamente a Dispose
```

Tabla comparativa: Java vs C# en liberación de recursos

Concepto	Java	C#	Error común a evitar
Recursos gestionados	try-with-resources	using	Olvidar cerrar en finally
Recursos no gestionados	try-finally	try-finally	No llamar a <code>Dispose()</code> manual
Cierre automático	<code>AutoCloseable</code>	<code>IDisposable</code>	Usar using donde no procede
Memoria nativa	N/A	<code>SafeHandle</code>	Llamar a <code>GC.Collect()</code> (no sirve)



Clave

Según el libro *Dependency Injection Principles* (Seemann, 2019), "todo servicio con estado debe implementar `IDisposable`".

7.1.3.4. Excepciones y Logging Estructurado

Este apartado es crítico para el sector público: cuando un sistema de pago de pensiones falla, no basta con saber que hubo un error. Necesitas saber exactamente qué pasó, dónde, con qué datos y cuándo, para cumplir con la auditoría del ENS y poder reconstruir la incidencia sin perder datos.

Problemas comunes que debes evitar

1. El error de perder el rastro de la excepción En C# existen dos formas de relanzar una excepción:

```
// ✗ MAL: pierdes el origen original del error
catch (Exception ex) { throw ex; }

// ✓ BIEN: conservas el stack trace completo
catch (Exception) { throw; }
```

El problema es sutil: cuando usas `throw ex;`, el compilador genera una nueva excepción y destruye la pila de llamadas original. En el portal de transparencia de Murcia, este error hizo imposible localizar la causa de 500 fallos diarios durante dos semanas.

2. Capturar excepciones genéricas

Esto es lo más peligroso que puedes hacer:

```
// ✗ MAL: atrapas desde un NIF mal formado hasta un fallo de memoria
catch (Exception e) { /*...*/ }
```



En su lugar, filtra por tipo usando *when*:

```
// ✅ BIEN: capturas solo lo que puedes manejar
catch (SqlException ex) when (ex.Number == 2627)
{
    // Solo entra aquí si es un error de clave duplicada
    _logger.Warning("Intento de duplicación en tabla {Tabla}", tabla);
}
```

El operador *when* es tu mejor aliado: evalúa condiciones sin entrar en el *catch*, evitando que captures excepciones que no entiendes.

7.1.3.5. Delegados, Lambdas y Expresiones LINQ Básicas

Delegados: intercambia métodos como si fueran variables

Un delegado es un "puntero limpio" a un método. Útil cuando necesitas cambiar el comportamiento sin reescribir la clase:

```
// Guardo el método EsNIFValido en una variable
Func<string, bool> validador = EsNIFValido;

// Puedo pasarlo como parámetro
public void ProcesarLote(List<string> nifs, Func<string, bool> validador)
{
    foreach (var nif in nifs)
    {
        if (!validador(nif)) { /*...*/ } // Uso el método que me pasan
    }
}
```



Uso real en migraciones: En el SEPE, migraron 300.000 expedientes con distintas reglas de validación. En lugar de crear 3 clases, usaron un delegado por tipo:

```
ProcesarLote(nifsAntiguos, ValidadorLegacy);  
ProcesarLote(nifsNuevos, ValidadorRD11122018);
```

Lambdas: métodos cortos sin declarar

En lugar de escribir un método completo, creas una mini-función en línea:

```
// Lambda que filtra empleados activos  
var activos = empleados.Where(e => e.Activo);  
LINQ: consultas legibles pero lentas para masivo  
LINQ permite escribir como SQL sobre listas:
```

```
// Legible pero ineficiente para 100.000 registros (3x más lento)  
var lista = ctx.Empleados.Where(e => e.Categoria == "A1").ToList();
```

Regla de oro:

LINQ → APIs, consultas dinámicas, pocos datos

for → ETL masivo, batches de nóminas, ficheros de 50.000 registros

```
// ✅ BIEN: para procesamiento masivo  
var empleadosA1 = new List<Empleado>();  
foreach (var emp in ctx.Empleados.AsNoTracking())  
{  
    if (emp.Categoria == "A1") empleadosA1.Add(emp);  
}  
// AsNoTracking evita caché innecesario, el for reduce presión en GC
```



7.1.4. Java



Fuente:

https://en.wikipedia.org/wiki/File:Java_programming_language_logo.svg

Es un lenguaje de programación de propósito general, concurrente, orientado a objetos, que fue diseñado específicamente para tener tan pocas dependencias de implementación como fuera posible.

Su intención es permitir que los desarrolladores de aplicaciones escriban el programa una vez y lo ejecuten en cualquier dispositivo, lo que quiere decir que el código que es ejecutado en una plataforma no tiene que ser recompilado para correr en otra. Para ello hace uso de la máquina virtual de Java.

7.1.5. Javascript



Fuente:

https://commons.wikimedia.org/wiki/File:Javascript_badge.svg



JavaScript (JS) es un lenguaje de programación interpretado, dialecto del estándar ECMAScript.

Se define como orientado a objetos, basado en prototipos, imperativo, débilmente tipado y dinámico.

Aunque comparte muchas de las características y de las estructuras del lenguaje Java, fue desarrollado de forma independiente.

El lenguaje JavaScript puede interactuar con el código HTML, estando embebido en el código fuente de la página web, permitiendo a los programadores utilizar contenido dinámico para ejecutar acciones en el lado del cliente.

El lenguaje JavaScript es *opensource*, por lo que cualquier persona puede utilizarlo sin comprar una licencia.

7.1.6. PHP



Fuente:

<https://es.m.wikipedia.org/wiki/Archivo:PHP-logo.svg>

Es un lenguaje de código abierto muy popular especialmente adecuado para el desarrollo web y que puede ser incrustado en HTML. Fue creado inicialmente por el programador danés-canadiense Rasmus Lerdorf en 1994.

Aunque el desarrollo de PHP está centrado en la programación de *scripts* del lado del servidor, se puede utilizar para muchas otras cosas. Lo mejor de utilizar PHP es su extrema simplicidad para el principiante, pero a su vez ofrece muchas características avanzadas para los programadores profesionales.

7.1.7. Python



Fuente:

<https://commons.wikimedia.org/wiki/File:Python-logo-notext.svg>



Es un lenguaje de programación cuya filosofía hace hincapié en una sintaxis que favorezca un código legible. Se trata de un lenguaje de programación multiparadigma, ya que soporta orientación a objetos, programación imperativa y, en menor medida, programación funcional. Es un lenguaje interpretado, usa tipado dinámico y es multiplataforma.

Es multiparadigma, ya que soporta orientación a objetos, programación imperativa y, en menor medida, programación funcional.

Administrado por la Python Software Foundation, posee una licencia de código abierto, denominada Python Software Foundation License.

Puesto que Python tiene como objetivo ser un lenguaje de fácil lectura, su formato es visualmente ordenado y, en muchas ocasiones se utilizan palabras clave en inglés en lugar de símbolos como ocurre en otros lenguajes.

Por ejemplo, los operadores lógicos `!`, `||` y `&&` en Python se escriben NOT, OR y AND, respectivamente.

Python utiliza la indentación (sangrado) para delimitar la estructura del programa permitiendo establecer bloques de código (bucles, funciones, clases, etc.) antes de cada línea de órdenes pertenecientes al bloque, mejorando así la legibilidad del código fuente. Esto diferencia a Python de otros lenguajes de programación que mantienen como costumbre declarar los bloques mediante un conjunto de caracteres (normalmente entre llaves).

Cada programador decide si utilizar espacios en blanco o tabuladores, pero se recomienda no mezclarlos.

Debido al significado sintáctico de la sangría, cada instrucción debe estar contenida en una sola línea, pero si por legibilidad se divide la instrucción en varias líneas, hay que añadir una barra invertida al final de la línea, lo que indica que la instrucción continúa en la siguiente.



+ Info

El término indentación es de uso común en informática, es un anglicismo de la palabra inglesa *indentation*, no reconocido por la Real Academia Española (en la vigesimosegunda edición), que recomienda utilizar el término "sangrado", que significa mover un bloque de texto hacia la derecha insertando espacios o tabuladores, para así separarlo del margen izquierdo y distinguirlo mejor del texto adyacente.



Los comentarios (que no son tenidos en cuenta por el intérprete) se pueden indicar de dos formas:

- Utilizar la notación `''' comentario '''`, tres apóstrofes de apertura y tres de cierre. Esta es la opción más recomendable.
- Utilizar el símbolo `#`, delante del comentario, que se interpretará como tal hasta el final de la línea.

Las variables se definen de forma dinámica, es decir, que no es necesario especificar cuál es su tipo de antemano y puede tomar distintos valores en otro momento, incluso de un tipo diferente al que tenía previamente. Para asignar valores se utiliza el símbolo `=`.



+ Info

El intérprete de Python estándar incluye un modo interactivo, en el cual se escriben las instrucciones una a una en una especie de intérprete de comandos, pudiendo verse el resultado de su evaluación inmediatamente, lo que da la posibilidad de probar porciones de código en el modo interactivo antes de integrarlo como parte de un programa.

En Python se utilizan las llaves para definir diccionarios y para formatear cadenas de texto incluyendo valores de variables o expresiones.

En las listas y tuplas los elementos se separan por comas, y pueden contener elementos de diferentes tipos, pero existen diferencias entre ambas:

- **Listas.**
 - Para declarar una lista se usan los corchetes `[ ]`.
 - Suelen usarse para elementos del mismo tipo en cantidad variable.
 - Se caracterizan por ser mutables, es decir, se puede cambiar su contenido en tiempo de ejecución.
- **Tuplas.**
 - Para declarar una tupla se usan los paréntesis `( )` y es necesario que tengan como mínimo una coma.
 - Suelen usarse para elementos distintos en cantidad fija.
 - Son inmutables ya que no es posible modificar el contenido una vez creada.



Para acceder a los elementos de una lista o tupla se utiliza un índice entero (empezando por "0", no por "1"). Se pueden utilizar índices negativos para acceder elementos a partir del final.

Ejemplo: variable prenda, en la que se almacena el tipo y los colores:

```
prenda = {'tipo': 'vestido', 'colores': ['rojo', 'verde', 'morado']}
```

¿Cómo asignaría a una variable llamada coloresSegundocolor el valor morado?

Respuesta:

```
coloresSegundocolor= prenda[colores][2]
```

Con índice negativo, sería:

```
coloresSegundocolor= prenda[colores][-1]
```

Python, permite hacer que una clase herede de varias superclases.



+ Info

Puedes obtener más información en la página oficial.

<https://www.python.org/>

Python para el Machine Learning

El Machine Learning (ML), traducido como Aprendizaje Automático, es una disciplina del campo de la Inteligencia Artificial que, a través de algoritmos, permite a los ordenadores tener la capacidad de identificar patrones en datos masivos y elaborar predicciones (análisis predictivo). Se automatiza eficientemente el proceso de creación de modelos analíticos y así las máquinas se adaptan a nuevas situaciones de manera independiente.



Python dispone de librerías que permiten el desarrollo de modelos de ML, como son:

- **Tensorflow:** una librería de aprendizaje automático de código abierto rápida, flexible y escalable para la investigación y la producción, disponible para trabajar con Machine Learning, creando modelos en ordenadores, dispositivos móviles y servidores (utilizando TensorFlow Lite y TensorFlow Serving). Proporcionado por Google.

Características son:

- Capacidades de abstracción.
 - Reconocimiento de imágenes, texto y voz.
 - Gestión de redes neuronales profundas.
 - Procesamiento del lenguaje natural.
 - Ecuación diferencial parcial.
- **Keras:** diseñado por un ingeniero de Google para ONEIROS (Open-Ended Neuro Electronic Intelligent Robot Operating System), fue rápidamente soportado en la librería principal de TensorFlow, haciéndolo accesible sobre TensorFlow.

Es una de las librerías de redes neuronales más populares y de código abierto para Python, con la tarea principal de crear modelos de aprendizaje profundo.

Keras amplía la usabilidad de TensorFlow, proporciona muchos de los bloques de construcción y herramientas necesarias para crear una red neuronal como:

- Capas neuronales.
 - Abandono.
 - Puesta en común.
 - Funciones de activación y de costes.
 - Objetivos.
 - Normalización por lotes.
- **PyTorch:** Desarrollado por Facebook, es una librería de aprendizaje automático para Python. (También soporta C++).

Considerado por muchos como framework de aprendizaje automático y aprendizaje profundo, es la competencia directa de TensorFlow.

- **Scikit-learn:** Incluye una fácil integración con varias librerías de programación ML como NumPy y Pandas. Se centra en el modelado de datos.



- **Pandas:** es una librería de análisis de datos de Python y se utiliza principalmente para la manipulación y el análisis de datos.
- **NumPy:** se centra en el manejo de grandes datos multidimensionales y de complejas funciones matemáticas que operan sobre los datos. Ofrece limpieza y manipulación de datos, una rápida computación y ejecución de complicadas funciones que trabajan sobre arrays.
- **NLTK:** significa Natural Language Toolkit, y es una librería de Python para trabajar con el procesamiento del lenguaje natural, tratamiento de textos.
- **Spark MLlib:** desarrollada por Apache, es la librería de aprendizaje automático escalable para Apache Spark, que permite escalar fácilmente sus cálculos.
- **Theano:** para definir, optimizar y evaluar fácilmente potentes expresiones matemáticas. Es una librería robusta para realizar cálculos científicos a gran escala.
- **MXNet:** de Apache, es una librería flexible y eficaz para el Deep Learning, es altamente escalable y permite un rápido entrenamiento de los modelos, utilizado para entrenar y desplegar redes neuronales profundas. Funciona también con una serie de otros lenguajes, como C++, Perl, Julia, R, Scala, Go y algunos otros. La portabilidad y escalabilidad de MXNet le permiten pasar de una plataforma a otra.
- **Matplotlib:** es una poderosa herramienta para graficar y visualizar datos. Está pensado para trazar gráficos de cualquier tipo (gráfico circular, histograma, gráfico de dispersión...). Contiene una sub-librería pyplot que crea una interfaz cercana al software comercial Matlab que contiene funciones muy similares a éste.
- **Seaborn:** basado en la librería Matplotlib, está totalmente adaptado a los DataFrames de Pandas para la visualización de datos, especializada en el análisis estadístico. Permite producir de forma rápida e intuitiva gráficos estadísticos de alta calidad.
- **Statsmodels:** librería que proporciona clases y funciones para estimar muchos modelos estadísticos diferentes, así como para realizar pruebas estadísticas y minería de datos estadísticos.
- **NetworkX:** librería para crear, manipular y estudiar la estructura, la dinámica y las funciones de redes complejas. Muy popular para el manejo de datos gráficos, ya que incluye muchas funciones que permiten la generación de gráficos y también múltiples características para leer y escribir gráficos en varios formatos.
- **BeautifulSoup:** librería para extraer datos de archivos HTML y XML, compatible con la mayoría de los navegadores. Se utiliza mucho para el web scraping, una técnica de extracción de contenidos de sitios web.
- **Openpyxl:** API de Python para leer y escribir formatos de archivo de Microsoft Excel 2010 (xlsx, xlsm, xltx y xltm). Es de código abierto.
- **Mahotas:** biblioteca de procesamiento de imágenes y visión para un entorno MacOS.



7.1.7.1. Machine Learning Services para SQL Server con Python y R

En este apartado vamos a tratar sobre Machine Learning Services qué es y cuál es su función dentro de SQL Server.



+ Info

Machine Learning Services, tal y como indica Microsoft "es una característica de SQL Server que proporciona la capacidad de ejecutar scripts de Python y R con datos relacionales. Para llevar a cabo análisis predictivo y aprendizaje automático, se pueden usar marcos y paquetes de código abierto, además de paquetes de Python y R de Microsoft. Los scripts se ejecutan en la base de datos sin mover los datos fuera de SQL Server o a través de la red. En este artículo se explican los conceptos básicos de SQL Server Machine Learning Services y cómo empezar a usarlo".

A continuación te facilitamos el enlace web donde podrás encontrar la Información completa: [¿Qué es Machine Learning Services para SQL Server con Python y R?](#)

Paquetes de Python y R

En el enlace indicado en el Autotexto veremos cómo además de existir paquetes de Python y R de código abierto preinstalados en Machine Learning Services podemos encontrar diversos paquetes de Microsoft en los mismos lenguajes.

A continuación citaremos textualmente una pincelada, obtenida en la misma página enlazada, sobre estos mismos:

Lenguaje Python:

- **Revoscalepy**: paquete principal para Python escalable. Transformaciones y manipulación de datos, resumen estadístico, visualización y muchas formas de modelado...
- **Microsoftml**: se aplica solamente a los SQL Server 2016, 2017 y 2019. Agrega algoritmos de aprendizaje automático para crear modelos personalizados dedicados al análisis de texto, imágenes y opiniones.



Lenguaje R:

- **RevoScaleR:** paquete principal para R escalable. Permite realizar transformaciones y manipulaciones de datos, resúmenes estadísticos, visualizaciones y muchas formas de modelado....
- **MicrosoftML:** se aplica solamente a los SQL Server 2016, 2017 y 2019. Agrega algoritmos de aprendizaje automático para crear modelos personalizados dedicados al análisis de texto, imágenes y opiniones.
- **OlapR:** se aplica solo a SQL Server 2016, SQL Server 2017 y SQL Server 2019. Se trata de funciones de R usadas para las consultas MDX en un cubo OLAP de SQL Server Analysis Services.
- **Sqlrutils:** se aplica solo a SQL Server 2016, SQL Server 2017 y SQL Server 2019. Este es un mecanismo para usar scripts de R en un procedimiento almacenado de T-SQL, registrar dicho procedimiento almacenado en una base de datos y ejecutarlo en un entorno de desarrollo de R.
- **Microsoft R Open:** (retirado) era la distribución mejorada de Microsoft R.

La información suministrada en esta página procede como se ha dicho de la biblioteca de documentación técnica de Microsoft, **Microsoft Learn**, conocido anteriormente como **Microsoft Docs**.

Asimismo, los enlaces de los paquetes citados te permitirán acceder a una información más detallada.

7.1.7.2. ADOdb

ADOdb es un conjunto de bibliotecas de bases de datos para PHP y Python.

Permite a los programadores desarrollar aplicaciones web de una manera portable, rápida y fácil. La ventaja reside en que la base de datos puede cambiar sin necesidad de reescribir cada llamada a la base de datos realizada por la aplicación.

ADOdb usa SQL (lenguaje de dominio específico), teniendo en cuenta que cada base de datos implementa SQL de una manera levemente diferente, es trabajo del desarrollador prestar cuidadosa atención a las características y funciones específicas de la base de datos para mantener la portabilidad del código.

ADOdb contiene componentes para consultar y actualizar bases de datos, así como una biblioteca de orientada a objetos, administración de esquemas y supervisión del rendimiento.

También contiene las siguientes extensiones independientes:

- Una biblioteca de fecha y hora para manejar fechas fuera de los límites normales de PHP.
- Una biblioteca de administración de sesiones que amplía la funcionalidad PHP normal para permitir el almacenamiento de datos de administración de sesiones en una base de datos, o en valores cifrados.



ADODB no es un reemplazo para las extensiones de base de datos PHP nativas, pero se basa en ellas. Esto significa que los controladores correspondientes deben estar instalados y configurados correctamente para que ADODB funcione.



+ Info

Puedes consultar más información en su web oficial:

<http://adodb.org/dokuwiki/doku.php>

7.1.8. Visual basic .Net (vb.Net)



Fuente:

https://commons.wikimedia.org/wiki/File:Microsoft_.NET_logo.png

Es un lenguaje de programación orientado a objetos que se puede considerar una evolución de Visual Basic implementada sobre el *framework* .NET.



7.1.9. Classic Visual Basic



Fuente: <https://pixabay.com/es/visual-basic-programaci%C3%B3n-idioma-906838/>

Es un lenguaje dirigido por eventos, desarrollado por Microsoft.

Este lenguaje de programación es un dialecto de BASIC, con importantes agregados cuya intención es simplificar la programación utilizando un IDE de desarrollo.

La última versión fue la 6, liberada en 1998, para la que Microsoft extendió el soporte hasta marzo de 2008. Aunque Visual Basic es de propósito general, también provee facilidades para el desarrollo de aplicaciones de bases de datos usando Data Access Objects, Remote Data Objects o ActiveX Data Objects.

7.1.10. SQL

SQL

SQL (*Structured Query Language* o lenguaje de consulta estructurado) es un lenguaje de dominio específico diseñado para administrar sistemas de gestión de bases de datos relacionales.

Una de sus principales características es el manejo del álgebra y el cálculo relacional para efectuar consultas con el fin de recuperar, de forma sencilla, información de bases de datos, así como realizar cambios en ellas.



7.1.11. PL/SQL



PL/SQL (*Procedural Language/Structured Query Language*) es un lenguaje de programación de procedimiento incrustado en Oracle. Soporta todas las consultas, ya que la manipulación de datos que se usa es la misma que en SQL, incluyendo nuevas características.

En un entorno de base de datos los programadores pueden construir bloques PL/SQL para utilizarlos como procedimientos o funciones, o bien pueden escribir estos bloques como parte de *scripts* SQL*Plus.

PL/SQL es una extensión del lenguaje SQL y se usa fundamentalmente en los sistemas de gestión de bases de datos Oracle. Permite definir funciones (que pueden devolver resultados) y procedimientos que aceptan argumentos, se almacenan compilados en las propias bases de datos y son invocados desde estas mismas. Otra de sus particularidades son los triggers o disparadores (pequeñas rutinas) que responden a determinados eventos de la base de datos (inserciones, actualizaciones, eliminaciones). Este lenguaje posee una estructura robusta que le permite manejar errores y excepciones.

7.2. Otros lenguajes de programación

Indicamos ahora algunos lenguajes menos conocidos, como:

- ASSEMBLY LANGUAGE (ASL).
- GO.
- MATLAB.
- PERL.
- R.
- Ruby.
- SCRATCH.
- Objective-C.
- Pascal.
- Delphi/Object Pascal.
- Groovy.
- Swift



Assembly language

Es un lenguaje de programación de bajo nivel.

Consiste en un conjunto de mnemónicos que representan instrucciones básicas para los computadores, microprocesadores, microcontroladores y otros circuitos integrados programables.

Go



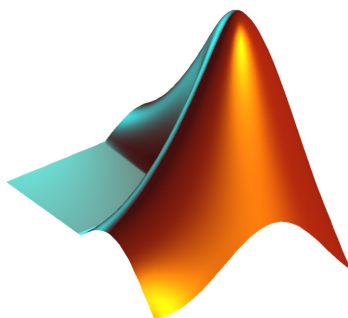
Fuente:
https://commons.wikimedia.org/wiki/File:Go_Logo_Aqua.svg

Es un lenguaje de programación concurrente y compilado inspirado en la sintaxis de C. Ha sido desarrollado por Google.

Actualmente está disponible en formato binario para los sistemas operativos Windows, GNU/Linux, FreeBSD y Mac OS X, pudiendo también ser instalado en estos y en otros sistemas con el código fuente.

Go es un lenguaje de programación compilado, concurrente, imperativo, estructurado, orientado a objetos (de una manera especial) y con recolector de basura. De momento está soportado en diferentes tipos de sistemas UNIX, incluidos Linux, FreeBSD y Mac OS X. Es un lenguaje de programación cada vez más popular para el desarrollo de backend.

Matlab



Fuente:
https://commons.wikimedia.org/wiki/File:Matlab_Logo.png



Matlab (*MATrix LABoratory*) es una herramienta de *software* matemático que ofrece un entorno de desarrollo integrado (IDE) con un lenguaje de programación propio (lenguaje M).

Está disponible para las plataformas Unix, Windows, Mac OS X y GNU/Linux. Es un *software* muy usado en universidades y centros de investigación y desarrollo.

Perl



Fuente:

https://commons.wikimedia.org/wiki/File:Cebolla_Chulita.png

Perl es un lenguaje de propósito general que toma características del lenguaje C, del lenguaje interpretado *bourne shell* (sh), AWK, sed, Lisp y, en un grado inferior, de muchos otros lenguajes de programación.

Estructuralmente, Perl está basado en un estilo de bloques como los del C o AWK y fue ampliamente adoptado por su destreza en el procesamiento de texto, así como por no tener ninguna de las limitaciones de los otros lenguajes de *script*.

R



Fuente:

https://commons.wikimedia.org/wiki/File:R_logo.svg

Es un entorno y lenguaje de programación con un enfoque al análisis estadístico. Se trata de uno de los lenguajes más utilizados en investigación por la comunidad estadística, siendo además muy popular en el campo de la minería de datos, la investigación biomédica, la bioinformática y las matemáticas financieras.



A esto contribuye la posibilidad de cargar diferentes bibliotecas o paquetes con funcionalidades de cálculo y gráficas. R es parte del sistema GNU y se distribuye bajo la licencia GNU GPL.

Está disponible para los sistemas operativos Windows, Macintosh, Unix y GNU/Linux.

Ruby



Fuente:
https://commons.wikimedia.org/wiki/File:Ruby_logo.svg

Es un lenguaje de programación interpretado, reflexivo y orientado a objetos. Combina una sintaxis inspirada en Python y Perl, con características de programación orientada a objetos similares a Smalltalk.

Comparte también funcionalidad con otros lenguajes de programación, como Lisp, Lua, Dylan y CLU. Ruby es un lenguaje de programación interpretado en una sola pasada y su implementación oficial es distribuida bajo una licencia de *software* libre.

Scratch



Imagen Scratchlogo.svg de Wikipedia

Scratch es un lenguaje de programación visual desarrollado por el Grupo Lifelong Kindergarten del MIT Media Lab.1.

Su principal característica consiste en que permite el desarrollo de habilidades mentales mediante el aprendizaje de la programación sin tener conocimientos profundos sobre el código.



Sus características ligadas al fácil entendimiento de la lógica de pensamiento de programación, han hecho que sea muy difundido en la educación de niños, adolescentes y adultos.

Diseñado para que todo el mundo pueda iniciarse en el mundo de la programación, crear historias interactivas, juegos y animaciones.

También facilitar la difusión de las creaciones finales con otras personas vía Web.

Objective-C

Objective-C

Es un lenguaje de programación orientado a objetos, creado como un supeconjunto de C para que implementase un modelo de objetos parecido al de Smalltalk. Actualmente se usa como un lenguaje principal de programación para Mac OS X, iOS y GNUstep, además de Swift.

Pascal

Pascal es un lenguaje de programación creado por el profesor suizo Niklaus Wirth entre los años 1968 y 1969 y publicado en 1970.

Su objetivo era crear un lenguaje para el aprendizaje de sus alumnos en programación estructurada y estructuración de datos.

Con el tiempo su utilización excedió el ámbito académico para convertirse en una herramienta para la creación de aplicaciones de todo tipo.

Pascal se caracteriza por ser un lenguaje de programación estructurado fuertemente tipado, por tanto:

- El código está dividido en porciones fácilmente legibles llamadas funciones o procedimientos.
- De esta forma Pascal facilita la utilización de la programación estructurada en oposición al antiguo estilo de programación monolítica.
- El tipo de dato de todas las variables debe ser declarado previamente para que su uso quede habilitado.



Delphi/Object Pascal

Delphi

Evolución del lenguaje de programación Pascal, que incluye elementos del paradigma orientado a objetos.

Es más conocido como el "lenguaje de programación de Borland Delphi", que modifica algunas características del Object Pascal original.

Borland vende entornos de desarrollo integrado (IDE) que compilan en lenguaje Delphi a Microsoft y Linux.

ASL

ASL

El lenguaje ensamblador (ASL o *Assembly Language*), es un lenguaje de programación de bajo nivel diseñado para interactuar con el *hardware* informático.

Implementa una representación simbólica de los códigos de máquina binarios y otras constantes necesarias para programar una arquitectura de procesador, y constituye la representación más directa del código máquina específico para cada arquitectura legible por un programador.

Un lenguaje ensamblador se utiliza para una arquitectura física (o virtual) específica. Esto contrasta con la mayoría de los lenguajes de programación de alto nivel, los cuales no dependen de la arquitectura física y, por lo tanto, son portables.

Groovy



Imagen: 1280px-Groovy-logo.svg de wikipedia

Es un lenguaje de programación orientado a objetos implementado sobre la plataforma Java.

Tiene características similares a Python, Ruby, Perl y Smalltalk.



La especificación JSR 241 se encarga de su estandarización para una futura inclusión como componente oficial de la plataforma Java.

Este lenguaje es de muy fácil adopción para programadores Java por las siguientes características:

- Groovy usa una sintaxis muy parecida a Java, comparte el mismo modelo de objetos, de hilos y de seguridad.
- Desde Groovy se puede acceder directamente a todas las API existentes en Java.
- La mayor parte de código escrito en Java es totalmente válido en Groovy.
- El bytecode generado en el proceso de compilación es totalmente compatible con el generado por el lenguaje Java para la Java Virtual Machine (JVM), por tanto, puede usarse directamente en cualquier aplicación Java.
- El aprendizaje es mucho más sencillo que en otros lenguajes que generan bytecode para la JVM, tales como Jython o JRuby.
- Groovy puede usarse también de manera dinámica como un lenguaje de scripting.



+ Info

Groovy 1.0 apareció el 2 de enero de 2007.

Después de varias versiones beta y otras tantas candidatas a release, el 7 de diciembre de 2007 apareció la versión Groovy 1.1 que finalmente fue renombrada a Groovy 1.5 con el fin de notar la gran cantidad de cambios que ha sufrido con respecto a la versión 1.0.

En diciembre de 2009 se publicó la versión 1.7.

Swift



Swift

Fuente:

https://commons.wikimedia.org/wiki/File:Swift_logo_with_text.svg

Es un lenguaje de programación multiparadigma creado por Apple y enfocado en el desarrollo de aplicaciones para iOS y macOS.



Está diseñado para integrarse con los *frameworks* Cocoa y Cocoa Touch, y puede usar cualquier biblioteca programada en Objective-C y llamar a funciones de C. También es posible desarrollar código en Swift compatible con Objective-C bajo ciertas condiciones.

Swift tiene la intención de ser un lenguaje seguro, de desarrollo rápido y conciso. En el año 2015 pasó a ser de código abierto.

8. Generaciones de los lenguajes de programación

Con la evolución de la informática a lo largo de los años, han ido surgiendo lenguajes de programación con características cada vez más sofisticadas y depuradas, lo que ha hecho que se realice una clasificación de los mismos en las llamadas 5 generaciones de lenguajes de programación, siendo:

- **Primera generación:**

Lenguaje máquina.

- **Segunda generación:**

Los lenguajes ensambladores.

- **Tercera generación:**

Se dice que en esta generación se crean los primeros lenguajes de alto nivel.

El trabajo de los programadores es más fácil, ya no tienen que concentrarse en la operación interna del procesador, disponen del uso de palabras claves (inglés) que hacen que una sola instrucción equivalga a muchas compiladas, y que además ya indican por sí mismas (o por su traducción al castellano) las operaciones que incluyen.

Ejemplo de esta generación son los lenguajes, por ejemplo, PASCAL, COBOL, BASIC, FORTRAN, C, etc.

- **Cuarta generación (4GL):**

Se les llama lenguajes orientados al usuario o RAD (Rapid Application Development, en castellano, desarrollo rápido de aplicaciones).

Son lenguajes capaces de generar código por sí solos, los usuarios finales pueden escribir sus propios programas de manera sencilla, por ejemplo, para generar consultas en una base de datos.

Se incluyen en esta generación los lenguajes orientados a objetos, con la posibilidad de reutilización de código para otros programas.

Ejemplo de esta generación son los lenguajes, por ejemplo, Visual, Natural Adabas.



- **Quinta generación:**

Los lenguajes naturales.

Aquí se encuentran los lenguajes orientados a la inteligencia artificial.

Estos lenguajes se asemejan más al lenguaje humano que los 4GL. Por ejemplo, cada vez hay más programas que responden a órdenes indicadas en lenguaje natural (por ejemplo, ChatGPT).

Ejemplo de esta generación es, por ejemplo, el lenguaje de programación LISP.

Es muy común en la actualidad el uso de chatbot para gestión de la atención a clientes y usuarios (centros de contacto, sistemas de respuesta interactiva (IVR)), pudiendo utilizarse también para estafas. Se trata de un programa de inteligencia artificial, capaz de simular una conversación con un usuario en lenguaje natural, como si se tratara de un ser humano, en aplicaciones de mensajería, sitios web, aplicaciones móviles o incluso telefónicamente.



Tenlo en cuenta

No hay una estandarización de esta clasificación de generaciones, pudiendo haber variaciones de dónde se engloba uno u otro programa (las fechas de los años) en función de los autores.

Puedes consultar una gráfica de tiempo en el siguiente enlace:

[Generaciones de los Lenguajes de Programación](#)

9. Bibliografía

- JOYANES AGUILAR, L. *Fundamentos de programación*. McGraw-Hill, 2008.
- <https://es.wikipedia.org>.
- <https://en.wikipedia.org>.
- <https://commons.wikimedia.org/>.
- <http://www.iqcelaya.itc.mx/~vicente/Programacion/Lenguajes.pdf>.
- <http://ocw.upm.es/ciencia-de-la-computacion-e-inteligencia-artificial/fundamentos-programacion/>.



- <http://www.iqcelaya.itc.mx/~vicente/Programacion/MainProgramacion.html>.
- <https://conceptobasicodecomputacion.weebly.com/lenguaje-de-alto-medio-y-bajo-nivel.html>.
- <https://es.slideshare.net/LyAndre/tipos-de-lenguaje-de-programacion>.
- <https://www.tiobe.com/tiobe-index/>.
- <https://definicion.de/>.
- [https://es.wikiversity.org/wiki/Fundamentos de programaci%C3%B3n](https://es.wikiversity.org/wiki/Fundamentos_de_programaci%C3%B3n).
- http://tic.taboadaleon.es/Unidad1-Programacion/Tema4_Herramientas/contenido/41_operadores.html.

